



REALITY CHECK: MULTIFORMAT DEEPPFAKE DETECTION SYSTEM



A PROJECT REPORT

Submitted by

Manikkam V

720322244011

In partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND BUSINESS SYSTEMS



AKSHAYA COLLEGE OF ENGINEERING AND TECHNOLOGY

(Autonomous)

Bhagavathipalayam, Kinathukadavu, Coimbatore,

Tamil Nadu – 642109.

MAY 2026

AKSHAYA COLLEGE OF ENGINEERING AND TECHNOLOGY

(Autonomous)

Bhagavathipalayam, Kinathukadavu, Coimbatore

Tamil Nadu-642109.

BONAFIDE CERTIFICATE

Certified that this project report "**REALITYCHECK: MULTIFORMAT DEEPFAKE DETECTION SYSTEM**" is the bonafide work of **Manikkam V (720322244011)** who carried out the project work under my supervision.

SIGNATURE

**DR. M. MALINI., M.Tech., Ph.D.,
HEAD OF THE DEPARTMENT**

Department of Computer Science
and Business Systems
Akshaya College of Engineering and
Technology, Kinathukadavu,
Coimbatore-642109

SIGNATURE

**DR. M. MALINI., M.Tech., Ph.D.,
SUPERVISOR**

ASSOCIATE PROFESSOR & HEAD
Department of Computer Science
and Business Systems
Akshaya College of Engineering and
Technology, kinathukadavu,
Coimbatore-642109

Submitted for project viva voce examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

With genuine humility, we are obediently thankful to God Almighty without Him, this work would have never become a reality.

We express our heartfelt thanks to our **Joint Director, Dr. N. SUGUNA, M.E., Ph.D.**, for her continuous encouragement, valuable support, and motivation throughout the course of this project.

We are deeply grateful to our **CEO, Dr. S. KAPILAN, M.E., Ph.D.**, whose visionary leadership and constant support have been a driving force in our academic and project endeavors.

We wish to express our sincere gratitude to our respected **Principal, Dr. R. RAVINDRAN, M. Tech., Ph.D.**, for providing us with the necessary resources and opportunity to successfully complete our project work.

We are thankful to our **Vice Principal, Dr. K. SIVASANKARI, M.E., Ph.D.**, for her insightful feedback and valuable guidance that contributed significantly to the progress of our project.

We extend our deep sense of gratitude to our beloved **DR. M. MALINI, M.Tech., Ph.D.**, Associate professor & Head, Department of Computer Science and Business Systems who has been a constant source of motivation and instrumental in broadening our knowledge and understanding.

Our sincere appreciation goes Project Guide, **DR. M. MALINI, M.Tech., Ph.D.**, Associate professor & Head, Department of Computer Science and Business Systems for her dedicated mentorship, timely feedback, and continuous support throughout the development of our project.

We also thank the project review members and all the faculty members of the Department of Computer Science and Business Systems for their valuable contributions to our learning journey.

ABSTRACT

The rapid growth of Artificial Intelligence has resulted in the advent of synthetic media commonly referred to as Deepfakes. These altered forms of media including but not limited to images, videos, audio files and text threaten not only the trustworthiness of information but also affect the cybersecurity of online resources and public confidence in their government as well as commercial institutions. Existing methods for detecting manipulated media are often informal and labor-intensive, making them ineffective against rapidly advancing deepfake technologies. As these techniques grow more sophisticated, manual approaches fail to scale efficiently. Therefore, there is a need for an automated, scalable system to detect synthetic media across multiple formats. This project develops web application using the Flask framework to allow users to upload any form of media content for authentication of authenticity via an automated interface. All submitted media types are processed through respective analysis pipelines ensuring that evaluation is consistent throughout the entire system. The system design is modular such that each media submission is first subject to application of feature extraction techniques specific to that media type, then the resultant feature values are classified by means of machine learning. Computer vision techniques are employed to analyse both image and video data; signal processing techniques are used to analyse audio data; and finally, natural language processing techniques are utilized on all text submissions. The listing of results for a given submission is retained in an SQLite database and made accessible to users at all times. The delivery of this fully integrated system allows for efficient and scalable methods to detect the presence of deepfake.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	iv
	LIST OF FIGURES	viii
	LIST OF ABBREVIATIONS	xi
1	INTRODUCTION	10
	1.1 Overview	10
	1.2 Problem Statement	11
	1.3 Hybrid Multimodal ML model	12
	1.4 Objectives of the Project	13
	1.5 Scope of the Project	14
2	LITERATURE REVIEW	15
3	SYSTEM ANALYSIS	20
	3.1 Existing System	20
	3.2 Proposed System	21
4	SYSTEM DESIGN	22
	4.1 General	22
	4.2 Hardware Requirements	22
	4.3 Software Requirements	23
	4.3.1 Flask Framework	23
	4.3.2 Multimodal Methods	23
	4.3.3 Data Processing Libraries	24
	4.3.4 Frontend Technologies	24

	4.3.5 SQLite Database	24
	4.3.6 Visual Studio Code	25
	4.3.7 Development Environment	25
	4.3.8 Web Browser	25
5	SYSTEM IMPLEMENTATION	26
	5.1 Project Description	26
	5.1.1 Methodology	27
	5.2 Module Description	28
	5.2.1 User Authentication Module	28
	5.2.2 Media Input Module	28
	5.2.3 Data PreProcessing Module	29
	5.2.4 Feature extraction Module	29
	5.2.5 Deep Fake Detection Module	29
	5.2.6 Database Management Module	30
	5.2.7 Administrative Monitoring Module	30
6	RESULTS AND DISCUSSION	31
	6.1 Result	31

7	CONCLUSION AND FUTURE ENHANCEMENT	35
	7.1 Conclusion	35
	7.2 Future Enhancement	36
	REFERENCES	37
	APPENDIX-1	39
	APPENDIX-2	59

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
5.1	Flow of the Project	27
6.1	Login Page	32
6.2	Register Page	33
6.3	Reality check Dashboard	34

LIST OF ABBREVIATIONS

ABBREVIATIONS	FULL FORM
AI	Artificial Intelligence
ML	Machine Learning
NLP	Natural Language Processing
TF-IDF	Term Frequency–Inverse Document Frequency
NB	Naive Bayes
STT	Speech-to-Text
TTS	Text to Speech
UI	User Interface
DB	Database
API	Application Programming Interface
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
GUI	Graphical User Interface
SVM	Support Vector Machine
ANN	Artificial Neural Network
CSV	Comma Separated Values
URL	Uniform Resource Locator
JSON	JavaScript Object Notation
IDE	Integrated Development Environment
DL	Deep Learning

CHAPTER-1

INTRODUCTION

1.1 OVERVIEW

The rapid growth of both digital content and artificial intelligence technologies has resulted in the rise in the creation and dissemination of deception through manipulated media. Deepfake content can consist of an altered image, video, audio, or text created by AI programs. Deepfake content may be used to spread false information, steal someone's identity, or commit fraud. The existing methods developed for detecting manipulated content are generally manual processes which lack consistency with their results, and thus, they are generally time-consuming and not practical for processing large volumes of digital content. These limitations ultimately impede the ability of people and organizations to authenticate media. With respect to the sophistication of the technology presently available to create deepfakes, there is an identified need for automated, accurate, and scalable deepfake detection systems.

This project is intended to create a Multi-format Deepfake Detection System using a Flask Web Framework to develop an integrated method for analyzing known manipulated media regardless of format. The developed system is designed to upload an image, video, audio file, or text and analyzing the item that has been uploaded. The methodology maximizes the accuracy of classification by quantifying the value of the features and using those values to classify or label the submitted items. The developed system will also maintain historical results related to submitted or processed items by maintaining the results in an SQLite database, as well as provide real-time feedback from a user using an interactive interface.

1.2 PROBLEM STATEMENT

While several techniques have been created for detecting manipulated digital content, there remain a number of limitations with the current detection methods regarding the following Accuracy, Scalability, Consistency. Most of the traditional detection methods rely on human evaluation and thus are subject to a bias and inconsistency relating to the assessment by the evaluator. As these deep fake technologies continue to expand and develop into more realistic formats, it will be even more difficult to differentiate between what is real and what is fake leading to unreliable results when data is verified using current methods.

At this point in time, there is no complete solution that enables users to access many types of media images, video, audio from one location. Most current and available technologies restrict themselves to detecting only one type of data. Therefore, no current detection technology can detect all types of images, video, audio, and text from a single interface. Because of this limitation, most detection technologies are not effective in real world situations where manipulated and altered content is presented in multiple formats. Because so many detection technologies do not support real time processing, they cannot produce a structured output and are inefficient for widespread usage.

The current lack of accessible tools prevents general users from effectively identifying manipulated digital content. Therefore, developing an automated and scalable multi-modal deepfake detection system is a critical necessity.

1.3 HYBRID MULTIMODAL MACHINE LEARNING METHODS

This system implements machine learning algorithms and employs media-specific analytical methodology to conduct intelligent identification of various types of deepfakes based on multiple media form factors. All processing occurs in real time and is accomplished using locally deployed web-based applications and the Flask framework, allowing operation both with and without external API connectivity to other services. The result is increased data confidentiality and privacy, decreased potential for service disruption or degradation due to reliance on external service dependencies and increased flexibility for executing in non-networked environments or in a controlled environment.

Each media type that is image, video, audio, or text will have dedicated detection pipelines incorporated into the overall architecture to enable full processing of all digital media types. The system uses advanced feature extraction techniques and machine learning classifiers to examine deeper characteristics of the media beyond simple pattern matching, including the identification of visual artefacts, audio discrepancies, and linguistic patterns manifesting in the affected media types. The combination of both structural and contextual characteristics enables the recognition of manipulated content through the analysis of both types.

A hybrid detection system will be used to increase the accuracy, reliability, and scalability of the detection algorithm by using distinct means of analyzing each media type's unique characteristics. In addition to this, the detection processes are designed to work together as a complete detection system. A variety of different analytic methods will be used to analyze and process the media-based deepfake content.

1.4 OBJECTIVES OF THE PROJECT

The objectives of the project are to design and develop an intelligent system capable of detecting deepfake content across multiple media formats using advanced artificial intelligence and machine learning techniques.

- To develop a web-based system for detecting deepfake content across images, videos, audio, and text.
- To implement machine learning models for accurate classification of media as real or fake.
- To extract and analyze features such as visual artifacts, audio patterns, and linguistic structures.
- To provide real-time detection results through an interactive and user-friendly dashboard.
- To store and manage detection results using an SQLite database for future reference and analysis.

Thus, the objective of this project is to develop a reliable, scalable, and intelligent system for effective multiformat deepfake detection.

1.5 SCOPE OF THE PROJECT

The multiformat system for detecting deepfakes is a web-based application that automates the identification of fake or altered digital content utilizing machine learning techniques. A single, consistent interface allows users to input and evaluate many different kinds of digital files (images, video audio, text) into the system at once. The system includes several ways to enter a validation process so that users can access the real world model of how to validate the authenticity of digital content with minimum effort and have access to an easy-to-use application.

This System is based on a modular architecture whereby different media formats are detected separately, The Images or videos are identified using computer vision techniques, audio using signal processing methods and text using natural language processing methods; this allows the system to identify deepfake data by using visual indicator, voice mismatches and writing patterns as points of identification. By using machine learning models, the system offers a higher level of accuracy in detecting whether or not media is real or fake than any previous system.

Users of the system will be able to access the system to get live results of their submissions and that results will be stored for future use in an SQLite database also to help manage users and keep track of overall system performance, there is an administrative module. The Multiformat Deepfake Detection System is written in Flask for back-end processing and employs HTML/CSS for frontend processing, therefore it can be built using highly scalable and developer friendly methodologies.

CHAPTER 2

LITERATURE REVIEW

[1] A Review of Deepfake Detection Techniques (2025)

A comprehensive survey on deepfake detection techniques was presented by S. Agarwal, T. Chakraborty, and P. K. Singh (2025), analyzing spatial, temporal, and biological signal-based approaches for identifying manipulated media. The study highlights machine learning and deep learning models such as CNNs for feature extraction and classification, along with benchmark datasets like FaceForensics++ and DFDC for evaluation. However, the paper identifies limitations such as poor generalization to unseen datasets, vulnerability to adversarial attacks, reliance on single-modality detection, and lack of unified multimodal frameworks, indicating the need for more scalable and robust detection systems.

[2] Multimodal Deepfake Detection using ViT and Diffusion Models (2025)

M. Javed, A. Rehman, and S. Saba (2025) proposed a multimodal framework integrating Vision Transformers and diffusion models to improve detection accuracy by capturing global features and subtle inconsistencies. The system enhances robustness through multimodal analysis but suffers from high computational complexity, large dataset requirements, increased processing time, and lack of optimization for real-time or edge deployment, limiting scalability and adaptability.

[3] Blockchain-based Framework for Secure Deepfake Detection (2025)

A blockchain-based deepfake detection framework was proposed by A. A. Khan and M. Zakarya (2025), which enhances security, transparency, and trust by integrating decentralized data management with detection mechanisms. The system uses blockchain technology to store and verify multimedia metadata, ensuring immutability and traceability of digital content. This approach allows users to validate the origin and integrity of media, reducing risks associated with tampering and misinformation. The framework is particularly useful in digital forensics and secure data sharing environments. However, the inclusion of blockchain introduces additional computational overhead and latency, making real-time detection challenging. The system also depends on network scalability and consensus mechanisms, and it focuses more on data security than improving detection accuracy, limiting its applicability in resource-constrained and real-time systems.

[4] Lightweight Multimodal Detection using ETMC and RawNet (2025)

Chakrapani D. S. and H. S. Saini (2025) proposed a lightweight multimodal deepfake detection framework that combines Efficient Temporal Multi-scale Convolution (ETMC) for video analysis and RawNet for audio signal processing. The system extracts temporal and spatial features from video frames while analyzing raw audio signals to detect synthetic speech patterns. By integrating visual and audio data, the framework improves detection accuracy and robustness against different types of deepfake attacks. Its lightweight design makes it suitable for faster processing and deployment in resource-limited environments. However, the model may compromise accuracy when handling highly sophisticated deepfakes, relies heavily on data quality and synchronization, lacks extensive evaluation across diverse datasets, and still faces challenges in achieving real-time performance in large-scale applications.

[5] Evolution of Generative Models From GANs to Diffusion (2025)

R. Babaei, S. Cheng, and S. Zhao (2025) presented a comprehensive review of generative models, focusing on the transition from Generative Adversarial Networks (GANs) to diffusion-based models. The study explains how GANs enabled realistic multimedia generation but faced challenges such as training instability and mode collapse. Diffusion models are introduced as a more stable and powerful alternative, capable of generating high-quality outputs through iterative noise removal processes. The paper provides insights into architectural differences, training mechanisms, and applications in deepfake creation. However, diffusion models require high computational resources and longer training times, making them unsuitable for real-time applications. Additionally, the study focuses more on generation rather than detection and highlights challenges related to scalability, deployment efficiency, and the increasing difficulty of detecting highly realistic synthetic content

[6] Audio Forensics and Synthetic Speech Detection (2025)

A study by A. O. M. Salih and T. A. Rashid (2025) focuses on detecting synthetic speech using advanced signal processing and deep learning techniques. The approach analyzes acoustic features such as frequency patterns, pitch variations, spectral characteristics, and temporal inconsistencies to differentiate between genuine and manipulated audio. Machine learning models are used to learn complex patterns in forged audio signals, enhancing detection accuracy. The system is useful in forensic analysis and security applications. However, it is limited to audio data and lacks multimodal integration with video and text. Its performance depends on high-quality audio input and may degrade in noisy environments. Additionally, challenges related to scalability, real-time processing, and adaptability to evolving speech synthesis technologies remain unresolved.

[7] Hybrid CNN-LSTM for Video Manipulation Detection (2025)

C. Sahu (2025) proposed a hybrid deep learning model that combines Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks to detect video manipulations. CNNs are used to extract spatial features such as textures and visual artifacts, while LSTM networks capture temporal dependencies across video frames. This combination allows the system to detect subtle inconsistencies in motion and frame continuity, improving detection accuracy. However, the model requires significant computational resources and longer training times. Its performance depends heavily on dataset quality and diversity, and it lacks optimization for real-time applications. Scalability and deployment in resource-constrained environments also remain challenges.

[8] Explainable AI for Transparent Deepfake Detection (2025)

N. Mansoor and A. I. Iliev (2025) proposed an explainable AI-based deepfake detection system that enhances transparency and interpretability of model predictions. The framework uses techniques such as saliency maps, attention visualization, and feature highlighting to explain how decisions are made. This improves user trust and supports better analysis in forensic applications. However, integrating explainability increases computational overhead and may impact system performance. The approach mainly focuses on visual data and lacks multimodal capabilities. It also faces challenges related to scalability, large dataset handling, and adaptation to evolving deepfake techniques.

[9] Systematic Review of Multimodal Fusion in AI Forensics(2026)

A. Suresh and V. Kumar (2026) conducted a systematic review of multimodal fusion techniques in AI forensics, emphasizing the integration of image, video, audio, and text data for improved deepfake detection. The study discusses early, late, and hybrid fusion strategies and highlights their role in enhancing accuracy and reducing false positives. Deep learning models are used to combine features from different modalities for better decision-making. However, the approach faces challenges such as high computational complexity, synchronization issues between data types, lack of standardized datasets, and difficulties in handling noisy or incomplete inputs. Scalability and real-time implementation also remain major concerns.

[10] Privacy-Preserving Forensics via Federated Learning (2025)

S. Singh and A. Dhumane (2025) proposed a federated learning-based framework for privacy-preserving deepfake detection, allowing decentralized model training without sharing sensitive data. The system aggregates locally trained models, ensuring data privacy and compliance with regulations. It supports secure and scalable model development across distributed environments. However, the approach introduces communication overhead, synchronization challenges, and inconsistencies due to varying data quality across devices. It also requires robust aggregation mechanisms and faces limitations in real-time deployment and scalability, indicating the need for more efficient and adaptive solutions.

CHAPTER 3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

Current approaches for detecting manipulated digital media primarily depend on manual verification and isolated detection techniques, which often result in inconsistent and inaccurate identification of deepfake content. Human-based evaluation relies heavily on individual perception and expertise, making it challenging to distinguish between authentic and manipulated media, especially as deepfake technologies continue to advance in realism. This subjective nature of analysis leads to unreliable outcomes and reduces the effectiveness of traditional verification methods. Furthermore, manual inspection is time-consuming and labor-intensive, making it impractical for processing large volumes of digital content efficiently.

In recent years, several automated deepfake detection tools have been developed, focusing on specific media formats such as images or videos. While these systems utilize machine learning and deep learning techniques, most are limited to single-format analysis and lack the ability to handle multiple types of media within a unified platform. Many of these solutions rely on predefined features or shallow analysis methods, which restrict their capability to detect highly sophisticated deepfakes. As a result, their performance often decreases when dealing with complex or unseen data.

In addition, current existing systems generally do not have the ability to process data as it comes in. They also do not provide an integrated manner to analyze images, videos, audio and texts formats together. Therefore, they are not as useful in practice and reflect the need for a comprehensive, scalable and efficient system for detecting deepfakes across all formats.

3.2 PROPOSED SYSTEM

The Multiformat Deepfake Detection System is a web application that detects manipulated digital media through machine learning and artificial intelligence techniques. The system provides a unified solution for analyzing multiple media formats (images, videos, audio files, and text) in one location. Users can upload content to be analyzed for authenticity via a user-friendly interactive interface that allows for efficient and consistent detection of manipulations.

As outlined before, each format of media undergoes processing techniques specific to that type of media. Digital images and video files utilize computer vision algorithms; audio files utilize signal processing techniques; and text files benefit from natural language processing algorithms. The various processing methods allow the detection of visual artifacts, vocal anomalies, and language characteristics associated with deepfake content and improve the accuracy of detecting these types of manipulated content.

In addition to providing results in real-time on whether the uploaded content was manipulated or authentic, the system archives detection results in an SQLite database which can be used for future analysis and reference. The system also includes an administrative module for monitoring system efficiency and managing user data. The system is developed using the Flask web framework for backend processing and HTML/CSS for frontend presentation to facilitate ease of use and scalability.

CHAPTER 4

SYSTEM DESIGN

4.1 GENERAL

The hardware and software components required for developing and implementing the Multiformat Deepfake Detection System are discussed in this chapter. The system is a web-based application that uses machine learning, computer vision, and natural language processing techniques to detect manipulated media such as images, videos, audio, and text. It provides real-time analysis and helps users verify the authenticity of digital content through an interactive interface. The hardware components support efficient system performance, while the software components are used for application development, media processing, model implementation, data storage, and system deployment.

4.2 HARDWARE REQUIREMENTS

The hardware requirements for the proposed Multiformat Deepfake Detection System are designed to support efficient execution of machine learning models, real-time analysis, and multimedia processing. A system with a multi-core processor such as Intel i5 or AMD Ryzen 5 is required to handle backend processing, media analysis, and model execution. A minimum of 8 GB RAM is recommended to ensure smooth performance while running the Flask server, processing user inputs, and performing detection tasks across different media formats.

Storage requirements include at least 256 GB of SSD storage to enable faster data access and efficient handling of uploaded media files. Since the system processes images, videos, audio, and text, sufficient storage is necessary to manage temporary and stored data effectively. Additionally, a stable internet connection is recommended for system updates and deployment, although the core detection processes can operate locally.

4.3 SOFTWARE REQUIREMENTS

The software components play a crucial role in the development, execution, and deployment of the Multiformat Deepfake Detection System. Each technology is selected based on its performance, efficiency, and compatibility with the overall system architecture, ensuring reliable media processing, accurate detection, and smooth integration of all system modules.

4.3.1 FLASK FRAMEWORK

The core functionality of the system is developed using Flask as the backend framework. Flask is responsible for handling user requests from the frontend, processing uploaded media files, and coordinating communication between different modules of the system. The backend manages file uploads, performs preprocessing, and passes the processed data to the respective machine learning models for deepfake detection.

4.3.2 MULTIMODAL DEEPPAKE DETECTION METHODS

Multimodal deepfake detection methods are used to analyze and classify digital media as real or fake across different formats such as images, videos, audio, and text. Separate machine learning models are applied for each media type, where image and video data are analyzed using computer vision techniques, audio data is processed using signal-based features, and text data is evaluated using natural language processing methods. These models examine patterns such as visual artifacts, voice inconsistencies, and linguistic irregularities to identify signs of manipulation. Each model generates predictions based on extracted features and contributes to the overall detection result. The system performs real-time analysis and provides a final classification indicating whether the content is authentic or manipulated. This multimodal approach improves detection accuracy by combining insights from different data types, making the system more reliable and effective in identifying complex deepfake content.

4.3.3 DATA PROCESSING LIBRARIES

Data processing libraries such as NumPy and Pandas are included in the system to handle and organize data efficiently for model processing. They help in cleaning, transforming, and structuring media-related features before passing them to machine learning models. These libraries enable efficient data manipulation and analysis, improving processing speed and ensuring accurate and consistent input for deepfake detection.

4.3.4 FRONTEND TECHNOLOGIES

The front-end of the application is built using HTML, CSS, and JavaScript, allowing users to upload media files such as images, videos, audio, or text for deepfake detection. The interface provides options to select the media type and view the analysis results. JavaScript enables dynamic interaction by handling API requests and displaying real-time detection results, giving users an interactive and smooth experience while using the system.

4.3.5 SQLITE DATABASE

User data, uploaded media details, and detection results are stored in an SQLite database that maintains records of all analyses for monitoring and future reference. This database provides an efficient way to store, retrieve, and manage data within the system, ensuring quick access to past detection results and supporting overall system functionality.

4.3.6 VS CODE

Visual Studio Code is the primary tool used to write and manage the source code for the Multiformat Deepfake Detection System. It provides features such as syntax highlighting, debugging support, and extension integration, which help improve developer productivity. It also enables efficient management of both frontend and backend code, keeping the workspace organized throughout the development process.

4.3.7 DEPLOYMENT ENVIRONMENT

The purpose of the deployment environment in this project is to run and manage the Multiformat Deepfake Detection System efficiently using tools like Visual Studio Code and a Flask local server. It ensures that the application and its dependencies operate smoothly within a consistent environment, reducing compatibility issues. This setup allows developers to easily write, test, debug, and modify the code during development. It supports quick deployment, reliable execution, and seamless integration of all system modules, making it suitable for both development and small-scale production use.

4.3.8 WEB BROWSER

A web browser such as Google Chrome or Microsoft Edge provides users with access to the Multiformat Deepfake Detection System and enables interaction with its features. Users can upload media files, initiate analysis, and view detection results directly through the browser. It supports all required functionalities, including user interaction, result visualization, and secure communication with the backend system.

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 PROJECT DESCRIPTION

Reality Check Multiformat Deepfake Detection System is an AI-based system designed to detect and analyze manipulated digital content such as images, videos, and text using machine learning, deep learning, and natural language processing techniques. The system aims to identify deepfakes and misinformation by evaluating visual, temporal, and semantic inconsistencies present in multimedia content. It provides users with a unified platform for verifying content authenticity and generating a credibility score based on model predictions. The architecture of the system is modular, where each component works together, including the user interface, media upload module, preprocessing unit, deepfake detection engine, multimodal fusion module, credibility scoring system, and result visualization module. The backend is developed using the Flask framework, which handles API requests, model integration, and communication between frontend and AI models.

The system supports multiple input formats such as images, videos, and text. Images are analyzed using deep learning models like CNNs, videos are processed through frame extraction and temporal analysis techniques, and text inputs are evaluated using NLP-based fake news detection models. The results from all modalities are combined using a fusion mechanism to generate a final authenticity score. The system also maintains a database such as SQLite to store user data, uploaded content, and detection results for future reference. Additionally, Docker can be used for containerization to ensure consistent deployment across different environments. Overall, the system provides an efficient, scalable, and intelligent solution for detecting deepfakes and ensuring digital content authenticity.

5.1.1 METHODOLOGY

This project proposes a systematic approach for detecting deepfake content using multimodal data and machine learning techniques. When a user uploads media image, video, audio, or text through the web interface, it is sent to the Flask backend for processing. The system performs preprocessing steps such as resizing images, extracting video frames, processing audio signals, and cleaning text data. Relevant features such as visual artifacts, audio patterns, and linguistic structures are then extracted and passed to machine learning models for analysis. This model evaluate these features to identify inconsistencies and generate a prediction indicating whether the content is real or fake. The detection results are stored in an SQLite database and displayed to the user through the interface in real time. This approach ensures efficient, accurate, and scalable deepfake detection across multiple media formats.

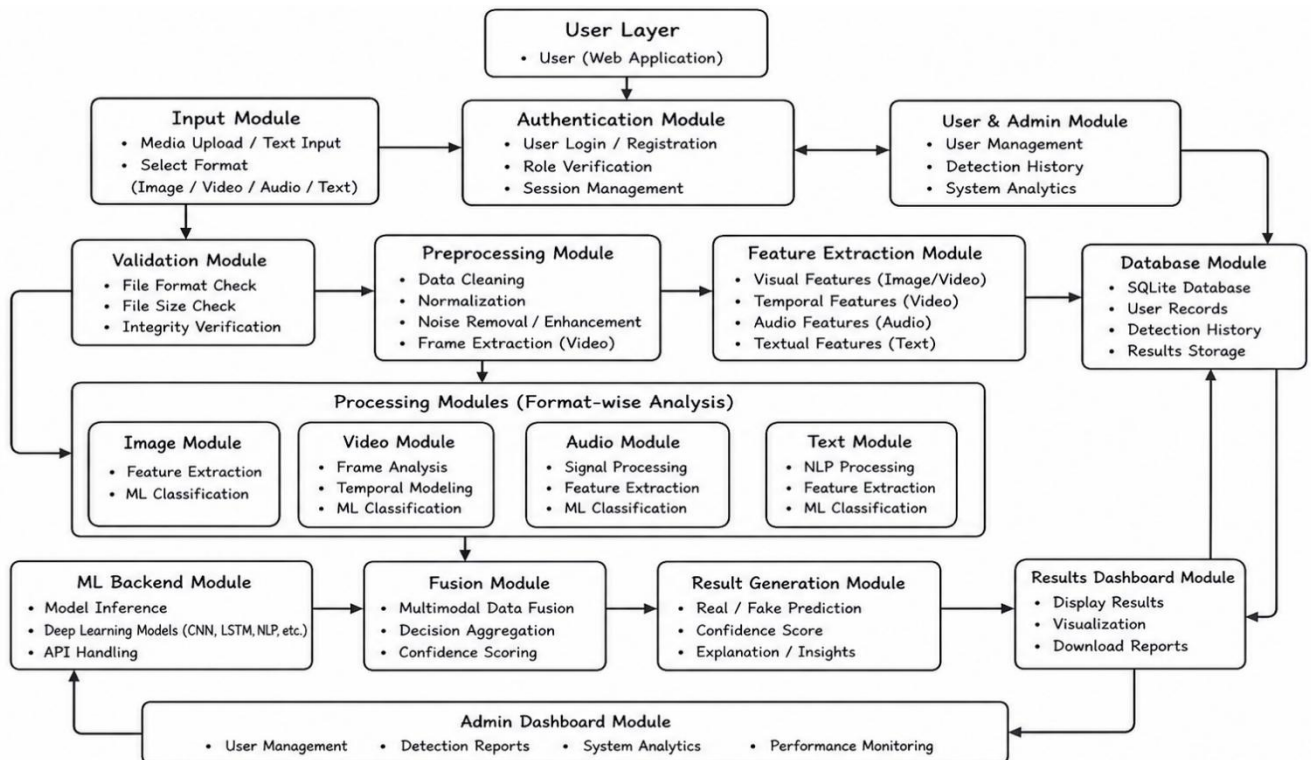


Figure 5.1 Flow of the Project

5.2 MODULE DESCRIPTION

5.2.1 USER AUTHENTICATION MODULE

The User Authentication Module is responsible for managing secure access to the Multiformat Deepfake Detection System. It allows users to register, log in, and interact with the platform through authenticated sessions. User credentials are securely handled using password hashing techniques, ensuring data protection. This module controls access to system features such as media upload, detection processing, and viewing of analysis results.

It ensures that only authorized users can access the system and manage their detection history. The module can also support role-based access, where administrators monitor system activity and manage data, while regular users perform detection tasks. By enforcing secure authentication and access control, this module helps maintain data privacy and overall system integrity.

5.2.2 MEDIA INPUT MODULE

The Media Input Module is responsible for collecting and handling user-uploaded media data for deepfake detection. It supports multiple input formats including images, videos, audio files, and text. This module validates the uploaded files based on type and size, ensuring they meet the system requirements before processing. It acts as the entry point for all data entering the system. Once the media is received, the module prepares it for further analysis by organizing and forwarding it to the preprocessing stage. For videos and audio files, it ensures proper extraction of frames or signals, while text inputs are directly passed for cleaning and analysis. This module plays a crucial role in enabling smooth and efficient data flow within the system.

5.2.3 DATA PREPROCESSING MODULE

The Data Preprocessing Module converts raw media inputs into a clean and standardized format suitable for machine learning models. It performs essential tasks such as resizing images, extracting frames from videos, filtering audio noise, and cleaning text by removing unnecessary characters. These steps ensure that the input data is consistent and ready for further analysis. This module also organizes and normalizes the processed data, making it suitable for feature extraction and model evaluation. By improving data quality and consistency, the preprocessing stage enhances the accuracy and efficiency of deepfake detection across different media formats.

5.2.4 FEATURE EXTRACTION MODULE

The Feature Extraction Module is responsible for converting preprocessed media data into meaningful features that can be used by machine learning models. It extracts important characteristics such as visual patterns from images and videos, audio features like MFCC from sound signals, and linguistic features from text data. These features help in identifying inconsistencies and patterns associated with deepfake content. This module ensures that the extracted data is structured and relevant for model analysis. By capturing key attributes from different media types, it improves the accuracy and efficiency of the detection process, making the system more reliable in identifying manipulated content.

5.2.5 DEEP FAKE DETECTION MODULE

The Deepfake Detection Module is the core component responsible for identifying whether the input media is real or fake. It uses machine learning models to analyze features extracted from images, videos, audio, and text. The module examines patterns such as visual artifacts, audio inconsistencies, and unnatural language

structures to detect signs of manipulation. Based on the analysis, the module generates a prediction indicating the authenticity of the content. It provides real-time results and ensures accurate classification by leveraging trained models, making the system reliable for detecting deepfake content across multiple media formats.

5.2.6 DATABASE MANAGEMENT MODULE

The Database Management Module is responsible for storing and managing all data generated within the Multiformat Deepfake Detection System. It uses SQLite to maintain records such as user details, uploaded media information, detection results, and timestamps. This ensures that all system data is organized and easily accessible for future reference. The module enables efficient data storage, retrieval, and updating of records, supporting smooth system operation. It also helps maintain a history of detection results, allowing users and administrators to review past analyses. By ensuring secure and structured data management, this module enhances the reliability and performance of the system.

5.2.7 ADMINISTRATIVE MONITORING MODULE

The Administrative Monitoring Module is responsible for overseeing the overall functioning and performance of the Multiformat Deepfake Detection System. It allows administrators to monitor user activity, review detection results, and analyze system performance metrics. This module provides insights into how the system is being used and helps identify patterns in detected deepfake content. It also enables administrators to manage stored data, track system usage, and ensure smooth operation of all modules. By providing centralized control and monitoring capabilities, this module helps maintain system reliability, improve performance, and support effective decisionmaking.

CHAPTER 6

RESULTS AND DISCUSSION

6.1 RESULTS

A Reality Check Multiformat Deepfake Detection System has been successfully developed and tested in a controlled environment to evaluate its efficiency, accuracy, and reliability in detecting manipulated digital content. The system is designed to simulate real-world content verification scenarios by supporting both image, video, and text-based inputs, allowing users to upload multimedia data and receive automated deepfake detection results without significant processing delay. This enables a smooth and efficient user experience for verifying the authenticity of digital content.

The implementation of machine learning, deep learning, and natural language processing techniques enables the system to perform contextual and feature-based analysis of input data. The combination of convolutional neural networks for image analysis, temporal modeling for video frames, and NLP-based fake news detection ensures reliable identification of inconsistencies, artifacts, and semantic anomalies. The multimodal fusion mechanism further improves accuracy by combining results from different data sources to generate a final authenticity score. Additionally, the system provides structured credibility reports that highlight whether content is real or manipulated along with confidence levels. The system also includes secure data handling, preprocessing, and administrative monitoring features to ensure smooth operation and controlled access. The results demonstrate that the proposed system is more efficient than traditional manual verification methods by providing automated, fast, and scalable deepfake detection. The findings support the feasibility of deploying such systems in social media monitoring, digital forensics, cybersecurity, and academic research environments, with strong potential for real-time large-scale implementation.

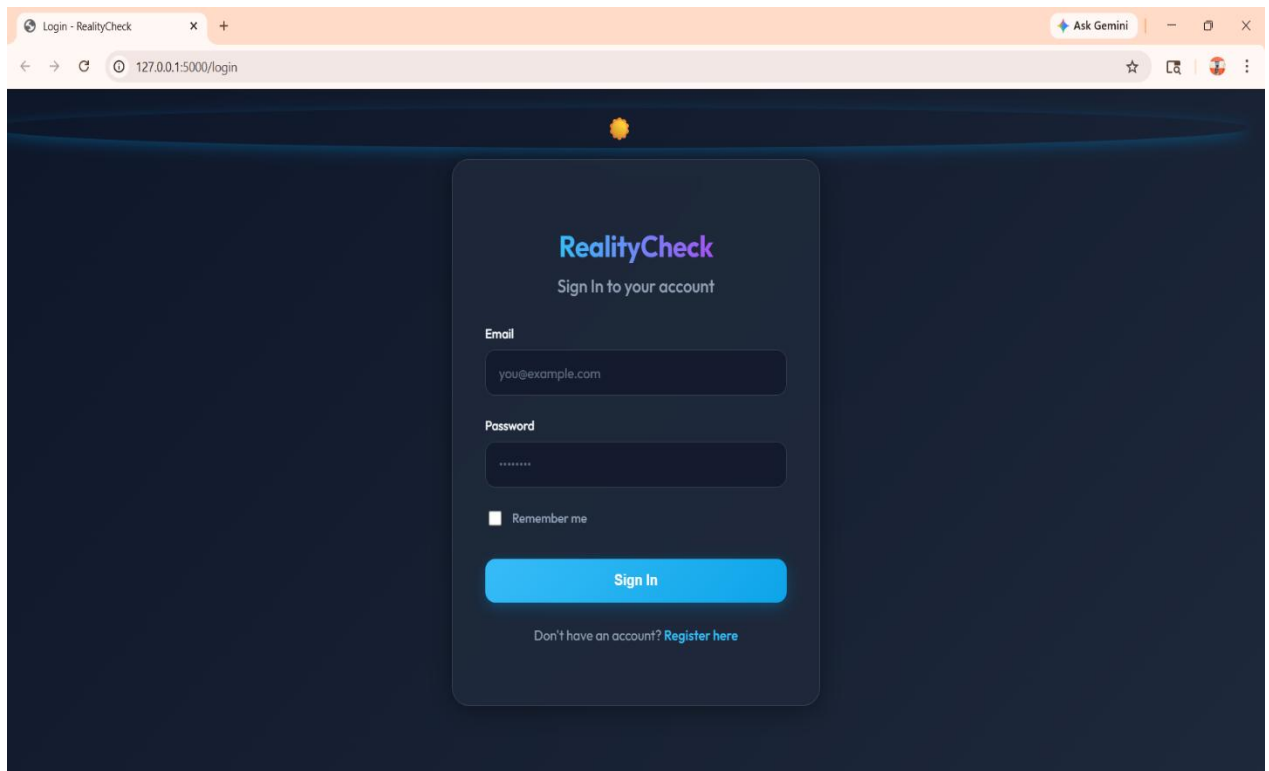


Figure 6.1 Login Page

The Figure 6.1 shows the Login Page of the Multiformat Deepfake Detection System. It provides a secure interface for users to enter their email and password to access the system. The page also includes options such as Remember me for user convenience and a link for new users to register an account. This ensures controlled and authenticated access to the system features, maintaining security and user data protection.

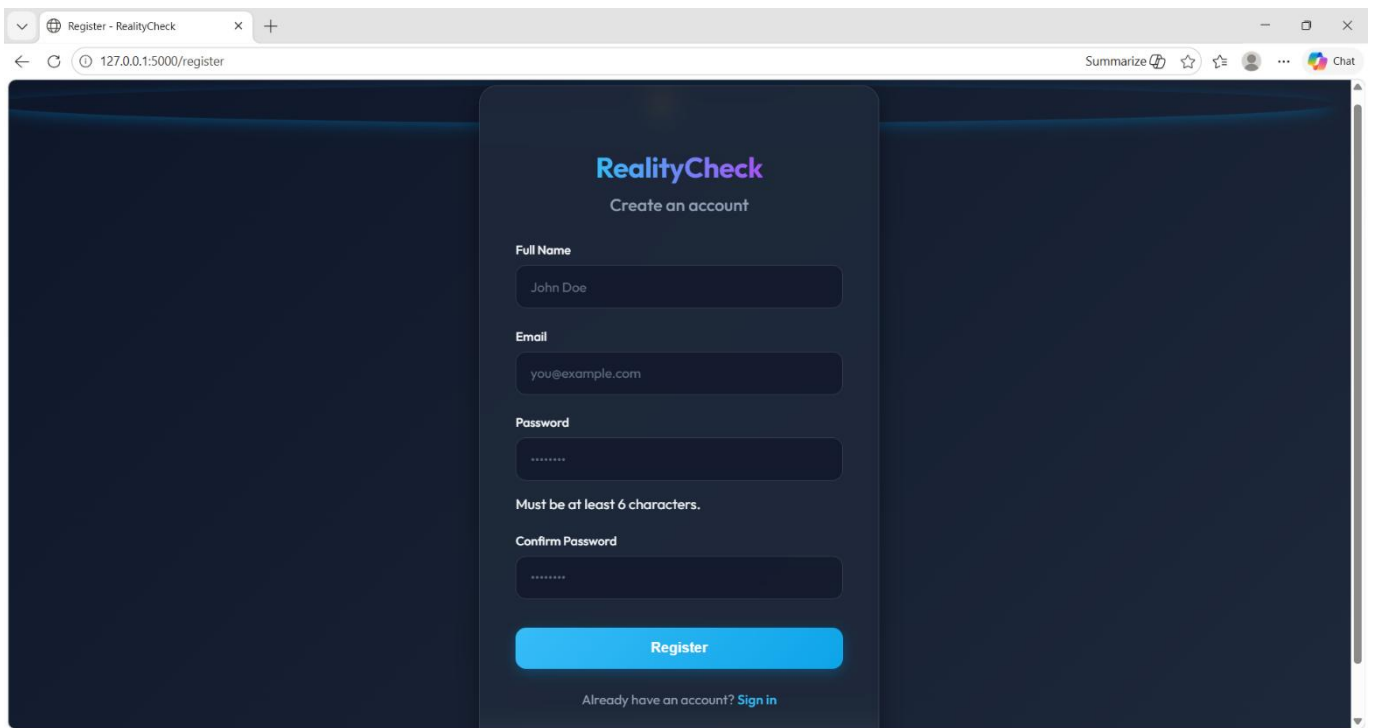


Figure 6.2 Register page

The Figure 6.2 describes the Register Page of the Multiformat Deepfake Detection System. It allows new users to create an account by entering details such as full name, email, and password. The page also includes password confirmation for validation and ensures that basic requirements like minimum password length are met. Additionally, it provides a link for existing users to sign in. This page enables secure user registration and serves as the entry point for accessing system features.

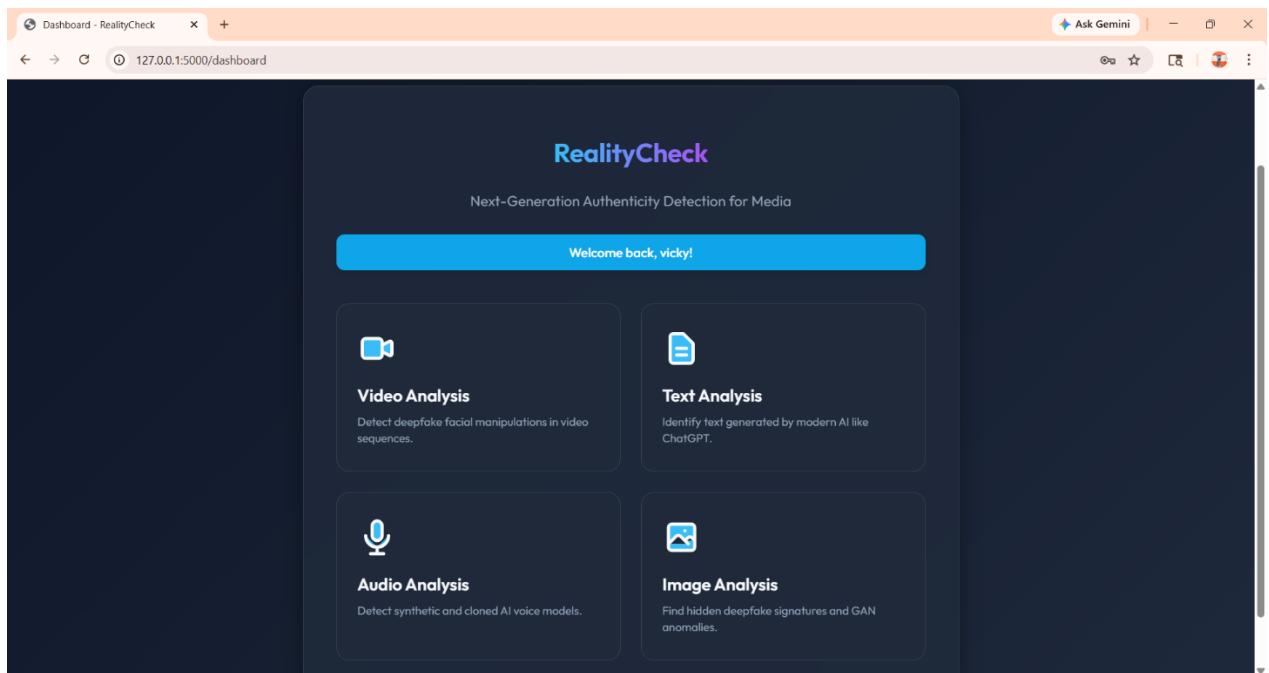


Figure 6.3 Reality Check Dashboard

The Figure 6.5 shows the Dashboard Page of the Multiformat Deepfake Detection System. It provides users with a centralized interface to access different analysis modules such as video, audio, image, and text detection. The dashboard displays a welcome message and presents options for selecting the type of media to analyze. This page enables users to easily navigate through the system and perform deepfake detection across multiple formats in a simple and interactive manner.

CHAPTER 7

CONCLUSION AND FUTURE EXHANCEMENT

7.1 CONCLUSION

This project demonstrates how the Reality Check: Multiformat Deepfake Detection System has been designed and implemented using web technologies, machine learning, deep learning, and natural language processing techniques to identify and analyze manipulated digital content. The system integrates multiple AI-based models for image, video, and text analysis, enabling automated detection of deepfakes and misinformation through a unified multimodal framework. It provides flexible input options where users can upload images, videos, or text-based content and receive real-time authenticity results along with credibility scores.

The proposed system addresses the limitations of traditional manual verification methods, which are often time-consuming, inconsistent, and prone to human error. By automating the detection process, the system ensures objective, fast, and reliable analysis of digital media. It leverages CNN-based models for visual artifact detection, temporal analysis for video forgery identification, and NLP techniques for detecting fake news and semantic inconsistencies. The multimodal fusion approach further enhances accuracy by combining outputs from different data sources into a final prediction score.

The system also improves transparency by providing structured detection reports that help users understand whether content is real or manipulated. Overall, the successful implementation of this system demonstrates its scalability, reliability, and practical applicability in areas such as social media monitoring, digital forensics, cybersecurity, and misinformation detection, contributing to the development of intelligent and automated content verification systems in the digital era.

7.2 FUTURE ENHANCEMENT

Future improvements to the Multiformat Deepfake Detection System will focus on enhancing accuracy, robustness, and real-time performance by integrating advanced deep learning and transformer-based models trained on updated datasets. The system can be extended to support real-time detection for live video and audio streams, enabling instant verification of content. Deploying the platform on cloud and edge environments will improve scalability and reduce latency across devices. Additionally, incorporating technologies such as digital watermarking, blockchain-based media tracking, and identity verification can further strengthen security, traceability, and trust in digital media.

REFERENCES:

1. N. S. A. Sharawardi, S. M. Sajjad, and Z. Muhammad, "Review of Deepfake Detection Techniques and Challenges," *Journal of Digital Forensics*, vol. 14, no. 1, pp. 45-52, Jan. 2025.
2. M. Javed, A. Rehman, and S. Saba, "Multimodal Deepfake Detection using ViT and Diffusion Models," *IEEE Access*, vol. 13, pp. 1102-1115, Feb. 2025.
3. A. A. Khan and M. Zakarya, "Blockchain-based Framework for Secure Deepfake Detection," *Journal of Cybersecurity*, vol. 8, no. 2, pp. 88-96, Feb. 2025.
4. Chakrapani D S and H. S. Saini, "Lightweight Multimodal Detection using ETMC and RawNet," *Multimedia Tools and Appl.*, vol. 32, no. 4, pp. 312-320, Aug. 2025.
5. R. Babaei, S. Cheng, and S. Zhao, "Evolution of Generative Models: From GANs to Diffusion," *AI Review*, vol. 19, no. 5, pp. 567-575, Oct. 2025.
6. A. O. M. Salih and T. A. Rashid, "Audio Forensics and Synthetic Speech Detection," *IEEE/CAA J. Autom. Sinica*, vol. 12, no. 2, pp. 150-158, Feb. 2025.
7. C. Sahu, "Hybrid CNN-LSTM for Video Manipulation Detection," *Int. J. Eng. Res. Technol. (IJERT)*, vol. 14, no. 3, pp. 201-209, Mar. 2025.

8. N. Mansoor and A. I. Iliev, "Explainable AI for Transparent Deepfake Detection," *Appl. Sci.*, vol. 15, no. 1, pp. 112-120, Jan. 2025..
9. A. Suresh and V. Kumar, "Systematic Review of Multimodal Fusion in AI Forensics," *Comput. Sci. Rev.*, vol. 22, pp. 104-112, Jan. 2026.
- 10.S. Singh and A. Dhumane, "Privacy-Preserving Forensics via Federated Learning," *IJSREM*, vol. 9, no. 4, pp. 441-449, Sep. 2025.

APPENDIX 1:

```
import os
from flask import Flask
from extensions import db, login_manager
from main_routes import main_blueprint
from auth_module.auth_routes import auth_blueprint
from auth_module.models import User
from video_module.video_routes import video_blueprint
from text_module.text_routes import text_blueprint
from audio_module.audio_routes import audio_blueprint
from image_module.image_routes import image_blueprint
from auth_module.profile_routes import profile_blueprint

app = Flask(__name__)
app.config['SECRET_KEY'] = os.environ.get('SECRET_KEY', 'dev-secret-key-change-
in-production')
app.config['SQLALCHEMY_DATABASE_URI'] = os.environ.get('DATABASE_URL', 'sqlite:///realitycheck.db')
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

app.config['UPLOAD_FOLDER'] = 'uploads/videos'
app.config['FRAME_FOLDER'] = 'uploads/frames'
app.config['AUDIO_FOLDER'] = 'uploads/audio'
app.config['IMAGE_FOLDER'] = 'uploads/images'
app.config['AVATAR_FOLDER'] = os.path.join('static', 'avatars')

os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)
os.makedirs(app.config['FRAME_FOLDER'], exist_ok=True)
os.makedirs(app.config.get('AUDIO_FOLDER', 'uploads/audio'), exist_ok=True)
os.makedirs(app.config.get('IMAGE_FOLDER', 'uploads/images'), exist_ok=True)
os.makedirs(app.config.get('AVATAR_FOLDER', os.path.join('static', 'avatars')),
exist_ok=True)

db.init_app(app)
login_manager.init_app(app)
login_manager.login_view = 'auth.login'
login_manager.login_message = 'Please log in to access this page.'

@login_manager.user_loader
def load_user(user_id):
```

```

    return db.session.get(User, int(user_id))

app.register_blueprint(main_blueprint)
app.register_blueprint(auth_blueprint)
app.register_blueprint(video_blueprint)
app.register_blueprint(text_blueprint)
app.register_blueprint(audio_blueprint)
app.register_blueprint(image_blueprint)
app.register_blueprint(profile_blueprint)

with app.app_context():
    db.create_all()
    try:
        rows = db.session.execute(db.text("PRAGMA table_info(user)").mappings().all())
        existing_cols = {r.get("name") for r in rows}
        if "mobile_no" not in existing_cols:
            db.session.execute(db.text("ALTER TABLE user ADD COLUMN mobile_no
VARCHAR(20)"))
        if "avatar" not in existing_cols:
            db.session.execute(db.text("ALTER TABLE user ADD COLUMN avatar
VARCHAR(256)"))
        db.session.commit()
    except Exception:
        db.session.rollback()

if __name__ == '__main__':
    import webbrowser
    import threading
    import time
    def open_browser():
        time.sleep(1.5)
        webbrowser.open('http://127.0.0.1:5000/login')
    if os.environ.get('WERKZEUG_RUN_MAIN') == 'true' or not app.debug:
        threading.Thread(target=open_browser, daemon=True).start()
    app.run(debug=True)

```



```

def _agent_log(hypothesis_id, location, message, data):
    # region agent log
    payload = {
        "sessionId": "6515f9",
        "runId": "pre-fix-video",
        "hypothesisId": hypothesis_id,
        "location": location,
        "message": message,
        "data": data,
        "timestamp": int(time.time() * 1000),
    }
    try:
        with open(DEBUG_LOG_PATH, "a", encoding="utf-8") as f:
            f.write(json.dumps(payload) + "\n")
    except Exception:
        pass
    # endregion

def predict_video_authenticity(frames):
    """
    Predict video authenticity by analyzing extracted frames.
    Uses image model on each frame and majority vote + average confidence.
    """
    # region agent log
    _agent_log("H2", "video_model.py:predict_video_authenticity", "Function entry",
{"frames_count": len(frames) if frames else 0})
    # endregion
    if not frames:
        return "Inconclusive", 50.0, {"reason": "No frames extracted"}

    try:
        from image_module.image_model import predict_image_authenticity
        # region agent log
        _agent_log("H3", "video_model.py:import", "Imported image model directly", {})
        # endregion
    except ImportError:
        import sys
        sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
        from image_module.image_model import predict_image_authenticity
        # region agent log
        _agent_log("H1_H3", "video_model.py:import", "ImportError branch taken with

```

```

local os import", {})
    # endregion

    results = []
    confidences = []
    per_frame = []
    for frame_path in frames[:30]: # max 30 frames
        # region agent log
        _agent_log("H1", "video_model.py:loop", "About to check frame path",
{"frame_path": str(frame_path)[:120]})
        # endregion
        if os.path.exists(frame_path):
            image_out = predict_image_authenticity(frame_path)
            res, conf = image_out[0], image_out[1]
            is_fake = "fake" in res.lower()
            results.append(1 if is_fake else 0)
            confidences.append(conf)
            per_frame.append({"fake": is_fake, "conf": conf})

    if not results:
        return "Inconclusive", 50.0, {"reason": "No valid frames processed"}

    fake_ratio = sum(results) / len(results)
    avg_conf = sum(confidences) / len(confidences)
    strong_fake = [x for x in per_frame if x["fake"] and x["conf"] >= 75]
    strong_fake_ratio = len(strong_fake) / len(per_frame)

    # Require stronger frame-level consensus before marking fake.
    if fake_ratio >= 0.7 and strong_fake_ratio >= 0.55:
        result = "Fake"
        confidence = round(((fake_ratio + strong_fake_ratio) / 2) * avg_conf, 2)
    else:
        result = "Inconclusive" if fake_ratio >= 0.5 else "Real"
        confidence = round((1 - fake_ratio) * avg_conf, 2)

    # region agent log
    _agent_log(
        "H2",
        "video_model.py:return",
        "Computed final video result",
        {
            "result": result,

```

```

        "confidence": confidence,
        "processed_frames": len(results),
        "fake_ratio": round(fake_ratio, 3),
        "strong_fake_ratio": round(strong_fake_ratio, 3),
        "avg_conf": round(avg_conf, 2),
    },
)
# endregion
return result, min(confidence, 99), {
    "processed_frames": len(results),
    "fake_ratio": round(fake_ratio, 3),
    "strong_fake_ratio": round(strong_fake_ratio, 3),
    "avg_frame_confidence": round(avg_conf, 2),
    "decision_rule": "fake if fake_ratio>=0.7 and strong_fake_ratio>=0.55",
}

```

```

def extract_frames(video_path, frame_folder, frame_rate=30):

```

```

    """Extracts frames from the video every nth frame"""

```

```

    cap = cv2.VideoCapture(video_path)

```

```

    frames = []

```

```

    count = 0

```

```

    frame_index = 0

```

```

    while True:

```

```

        success, frame = cap.read()

```

```

        if not success:

```

```

            break

```

```

        if count % frame_rate == 0:

```

```

            frame_name = f"frame_{frame_index}.jpg"

```

```

            frame_path = os.path.join(frame_folder, frame_name)

```

```

            cv2.imwrite(frame_path, frame)

```

```

            frames.append(frame_path)

```

```

            frame_index += 1

```

```

        count += 1

```

```

    cap.release()

```

```

    return frames

```

```

from .video_processing import extract_frames

```

```
from .video_model import predict_video_authenticity
from .utils import allowed_file
```

```
DEBUG_LOG_PATH = 'debug-6515f9.log'
```

```
def _agent_log(hypothesis_id, location, message, data):
    # region agent log
    payload = {
        "sessionId": "6515f9",
        "runId": "pre-fix-video",
        "hypothesisId": hypothesis_id,
        "location": location,
        "message": message,
        "data": data,
        "timestamp": int(time.time() * 1000),
    }
    try:
        with open(DEBUG_LOG_PATH, "a", encoding="utf-8") as f:
            f.write(json.dumps(payload) + "\n")
    except Exception:
        pass
    # endregion
```

```
video_blueprint = Blueprint('video', __name__, url_prefix='/video')
```

```
@video_blueprint.route('/')
@login_required
```

```
def index():
```

```
    return render_template('video.html')
```

```
@video_blueprint.route('/upload', methods=['POST'])
@login_required
```

```
def upload_video():
```

```
    try:
```

```
        # region agent log
        _agent_log("H4", "video_routes.py:upload_video", "Upload route entered",
```

```
{ "has_video_key": 'video' in request.files })
```

```
        # endregion
```

```
        if 'video' not in request.files:
```

```
            return render_template('video.html', error='No video file provided')
```

```
        file = request.files['video']
```

```

if file.filename == "":
    return render_template('video.html', error='No file selected')

if not allowed_file(file.filename):
    return render_template('video.html', error='Invalid file format. Use MP4, AVI,
MOV, or MKV')

upload_folder = current_app.config['UPLOAD_FOLDER']
os.makedirs(upload_folder, exist_ok=True)
filepath = os.path.join(upload_folder, file.filename)
file.save(filepath)

frame_folder = current_app.config['FRAME_FOLDER']
os.makedirs(frame_folder, exist_ok=True)
frames = extract_frames(filepath, frame_folder)
# region agent log
    _agent_log("H2", "video_routes.py:upload_video", "Frames extracted",
{"frames_count": len(frames) if frames else 0})
# endregion

out = predict_video_authenticity(frames)
result, confidence = out[0], out[1]
details = out[2] if len(out) > 2 else {}
    return render_template('result.html', result=result, confidence=confidence,
media_type='Video', details=details)
except Exception as e:
    # region agent log
    _agent_log("H1_H4", "video_routes.py:upload_video", "Route exception", {"error":
str(e)})
    # endregion
    current_app.logger.exception("Video analyze error")
    return render_template('video.html', error=f'Analysis failed: {str(e)}')

def highlight_fake_frames(frames, output_folder):
    """
    Draws red borders around frames suspected as fake.
    (This is a placeholder visualization)
    """
    os.makedirs(output_folder, exist_ok=True)
    highlighted = []

```

```

for frame_path in frames:
    img = cv2.imread(frame_path)
    if img is None:
        continue
    height, width, _ = img.shape
    cv2.rectangle(img, (10, 10), (width-10, height-10), (0, 0, 255), 5)
    out_path = os.path.join(output_folder, os.path.basename(frame_path))
    cv2.imwrite(out_path, img)
    highlighted.append(out_path)

```

```

return highlighted

```

```

def _get_audio_pipeline():
    global _AUDIO_PIPELINE
    if _AUDIO_PIPELINE is None:
        from transformers import pipeline
        _AUDIO_PIPELINE = pipeline(
            "audio-classification",
            model="MelodyMachine/Deepfake-audio-detection-V2"
        )
    return _AUDIO_PIPELINE
_N_MFCC = 40
_MAX_LEN = 100

```

```

def _extract_mfcc(audio_path):
    """Extract MFCC features from audio file."""
    try:
        import librosa
        y, sr = librosa.load(audio_path, sr=22050, duration=5.0)
        mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=_N_MFCC, n_fft=2048,
hop_length=512)
        mfcc = np.mean(mfcc.T, axis=0) # mean over time
        if len(mfcc) < _N_MFCC:
            mfcc = np.pad(mfcc, (0, _N_MFCC - len(mfcc)))
        return mfcc[:_N_MFCC].reshape(1, _N_MFCC, 1)
    except Exception:
        return np.zeros((1, _N_MFCC, 1))

```

```

def _build_audio_model():
    import tensorflow as tf

```

```

from tensorflow.keras import layers, Model

inp = layers.Input(shape=(_N_MFCC, 1))
x = layers.Conv1D(32, 3, activation='relu')(inp)
x = layers.MaxPooling1D(2)(x)
x = layers.Conv1D(64, 3, activation='relu')(x)
x = layers.GlobalAveragePooling1D()(x)
x = layers.Dense(32, activation='relu')(x)
x = layers.Dropout(0.3)(x)
x = layers.Dense(1, activation='sigmoid')(x)
return Model(inputs=inp, outputs=x)

def _get_audio_model():
    global _AUDIO_MODEL
    if _AUDIO_MODEL is None:
        model = _build_audio_model()
        weights_path = os.path.join(os.path.dirname(__file__), '..', 'models',
'audio_weights.keras')
        if os.path.exists(weights_path):
            model.load_weights(weights_path)
        _AUDIO_MODEL = model
    return _AUDIO_MODEL

def predict_audio_authenticity(audio_path):
    """
    Predict if audio is authentic or synthetic/deepfake.
    Uses MFCC + CNN (run train_audio_model.py first for best accuracy).
    """
    try:
        mfcc = _extract_mfcc(audio_path)
        weights_path = os.path.join(os.path.dirname(__file__), '..', 'models',
'audio_weights.keras')
        if not os.path.exists(weights_path):
            return _predict_with_hf_pipeline(audio_path)

        model = _get_audio_model()
        prob = float(model.predict(mfcc, verbose=0)[0][0])

        if 0.45 < prob < 0.6:
            return "Inconclusive", round(max(prob, 1 - prob) * 100, 2), {
                "model": "mfcc-cnn",
                "prob_fake": round(prob, 4),

```

```

        "reason": "Borderline score zone",
    }
    if prob >= 0.6:
        result = "Synthetic / Deepfake"
        confidence = round(prob * 100, 2)
    else:
        result = "Authentic"
        confidence = round((1 - prob) * 100, 2)

    return result, min(confidence, 99), {
        "model": "mfcc-cnn",
        "prob_fake": round(prob, 4),
        "reason": "MFCC + CNN prediction",
    }
except Exception as e:
    return _predict_with_hf_pipeline(audio_path)

def _predict_with_hf_pipeline(audio_path):
    """
    Use HuggingFace audio-classification pipeline for zero-shot detection.
    """
    try:
        pipe = _get_audio_pipeline()
        import librosa
        # Pipeline expects raw waveform at 16k usually, pipeline handles it if we pass path
        # or standard loading
        results = pipe(audio_path)
        # results e.g. [{"label": "fake", "score": 0.9}, {"label": "real", "score": 0.1}]
        fake_score = 0.0
        for r in results:
            if "fake" in r["label"].lower() or "spoof" in r["label"].lower():
                fake_score = float(r["score"])
                break

        if 0.45 < fake_score < 0.6:
            return "Inconclusive", round(max(fake_score, 1 - fake_score) * 100, 2), {
                "model": "MelodyMachine/Deepfake-audio-detection-V2",
                "prob_fake": round(fake_score, 4),
                "reason": "Borderline score zone",
            }
        if fake_score >= 0.6:
            return "Synthetic / Deepfake", round(fake_score * 100, 2), {

```



```

        "model": "MelodyMachine/Deepfake-audio-detection-V2",
        "prob_fake": round(fake_score, 4),
        "reason": "HF audio-classification prediction",
    }
    return "Authentic", round((1 - fake_score) * 100, 2), {
        "model": "MelodyMachine/Deepfake-audio-detection-V2",
        "prob_fake": round(fake_score, 4),
        "reason": "HF audio-classification prediction",
    }
except Exception:
    return _predict_with_spectral_heuristic(audio_path)

def _predict_with_spectral_heuristic(audio_path):
    """
    Spectral-based heuristic: synthetic voice often has different
    spectral envelope and harmonic structure.
    """
    try:
        import librosa
        y, sr = librosa.load(audio_path, sr=22050, duration=3.0)
        spec = np.abs(librosa.stft(y))
        spectral_flatness = np.mean(librosa.feature.spectral_flatness(S=spec))
        zero_crossings = np.mean(librosa.feature.zero_crossing_rate(y))
        # Synthetic audio often has different characteristics
        fake_score = min(1.0, (spectral_flatness * 2 + zero_crossings * 10) * 0.5)
        # Center around neutral; avoid fake-biased prior.
        fake_prob = np.clip(0.5 + (fake_score - 0.5) * 0.35, 0.1, 0.9)
        if 0.45 < fake_prob < 0.6:
            result = "Inconclusive"
        elif fake_prob >= 0.6:
            result = "Synthetic / Deepfake"
        else:
            result = "Authentic"
        confidence = round(abs(fake_prob - 0.5) * 120 + 55, 2)
        return result, min(confidence, 99), {
            "model": "spectral-heuristic",
            "prob_fake": round(fake_prob, 4),
            "reason": "Spectral flatness + zero crossings",
        }
    except Exception:
        return "Inconclusive", 50.0, {"model": "fallback", "reason": "Audio model
unavailable"}

```

```

audio_blueprint = Blueprint('audio', __name__, url_prefix='/audio')

@audio_blueprint.route('/')
@login_required
def index():
    return render_template('audio.html')

@audio_blueprint.route('/analyze', methods=['POST'])
@login_required
def analyze_audio():
    try:
        if 'audio' not in request.files:
            return render_template('audio.html', error='No audio file provided')

        file = request.files['audio']
        if file.filename == "":
            return render_template('audio.html', error='No file selected')

        if not allowed_file(file.filename):
            return render_template('audio.html', error='Invalid format. Use WAV, MP3, M4A, OGG, or FLAC')

        upload_folder = current_app.config.get('AUDIO_FOLDER', 'uploads/audio')
        os.makedirs(upload_folder, exist_ok=True)
        filepath = os.path.join(upload_folder, file.filename)
        file.save(filepath)

        out = predict_audio_authenticity(filepath)
        result, confidence = out[0], out[1]
        details = out[2] if len(out) > 2 else {}
        return render_template('result.html', result=result, confidence=confidence,
media_type='Audio', details=details)
    except Exception as e:
        current_app.logger.exception("Audio analyze error")
        return render_template('audio.html', error=f'Analysis failed: {str(e)}')

auth_blueprint = Blueprint('auth', __name__)

```

```

@auth_blueprint.route('/login', methods=['GET', 'POST'])
def login():
    if current_user.is_authenticated:
        return redirect(url_for('main.dashboard'))
    if request.method == 'POST':
        email = request.form.get('email', "").strip().lower()
        password = request.form.get('password', "")
        if not email or not password:
            flash('Please fill in both email and password.')
            return render_template('login.html')
        user = User.query.filter_by(email=email).first()
        if user and check_password_hash(user.password_hash, password):
            login_user(user, remember=request.form.get('remember', False))
            flash(f'Welcome back, {user.name}!')
            return redirect(url_for('main.dashboard'))
        flash('Invalid email or password.')
    return render_template('login.html')

```

```

@auth_blueprint.route('/register', methods=['GET', 'POST'])
def register():
    if current_user.is_authenticated:
        return redirect(url_for('main.dashboard'))
    if request.method == 'POST':
        name = request.form.get('name', "").strip()
        email = request.form.get('email', "").strip().lower()
        password = request.form.get('password', "")
        confirm = request.form.get('confirm_password', "")
        if not name or not email or not password or not confirm:
            flash('Please fill in all fields.')
            return render_template('register.html')
        if password != confirm:
            flash('Passwords do not match.')
            return render_template('register.html')
        if len(password) < 6:
            flash('Password must be at least 6 characters.')
            return render_template('register.html')
        if User.query.filter_by(email=email).first():
            flash('An account with this email already exists.')
            return render_template('register.html')
        user = User(
            name=name
            email=email,

```

```

        password_hash=generate_password_hash(password, method='pbkdf2:sha256')
    )
    db.session.add(user)
    db.session.commit()
    login_user(user, remember=False)
    flash('Account created successfully!')
    return redirect(url_for('main.dashboard'))
return render_template('register.html')

```

```

@auth_blueprint.route('/logout')
@login_required
def logout():
    logout_user()
    flash('You have been logged out.')
    return redirect(url_for('auth.login'))

```

```

profile_blueprint = Blueprint('profile', __name__, url_prefix='/profile')

```

```

ALLOWED_AVATAR_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif', 'webp'}

```

```

def _allowed_avatar(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in
ALLOWED_AVATAR_EXTENSIONS

```

```

def index():
    return render_template('profile.html', user=current_user)
def update():
    name = request.form.get('name', "").strip()
    email = request.form.get('email', "").strip()
    mobile_no = request.form.get('mobile_no', "").strip()
    current_password = request.form.get('current_password', "")
    new_password = request.form.get('new_password', "")
    confirm_password = request.form.get('confirm_password', "")

```

```

    if not name or not email:
        flash('Name and email are required.', 'error')
        return redirect(url_for('profile.index'))

```

```

# Check if email changed and already taken
from auth_module.models import User
if email != current_user.email:
    existing = User.query.filter_by(email=email).first()

```

```

    if existing:
        flash('That email is already in use.', 'error')
        return redirect(url_for('profile.index'))

current_user.name = name
current_user.email = email
current_user.mobile_no = mobile_no if mobile_no else None

# Handle avatar upload
avatar_file = request.files.get('avatar')
if avatar_file and avatar_file.filename:
    if not _allowed_avatar(avatar_file.filename):
        flash('Invalid file type. Use PNG, JPG, JPEG, GIF, or WEBP.', 'error')
        return redirect(url_for('profile.index'))
    avatar_folder = os.path.join(current_app.root_path, 'static', 'avatars')
    os.makedirs(avatar_folder, exist_ok=True)
    filename = f'user_{current_user.id}_{secure_filename(avatar_file.filename)}'
    avatar_file.save(os.path.join(avatar_folder, filename))
    current_user.avatar = filename

# Handle password change
if new_password or confirm_password or current_password:
    if not check_password_hash(current_user.password_hash, current_password):
        flash('Current password is incorrect.', 'error')
        return redirect(url_for('profile.index'))
    if new_password != confirm_password:
        flash('New passwords do not match.', 'error')
        return redirect(url_for('profile.index'))
    if len(new_password) < 6:
        flash('New password must be at least 6 characters.', 'error')
        return redirect(url_for('profile.index'))
    current_user.password_hash = generate_password_hash(new_password,
method='pbkdf2:sha256')

db.session.commit()
flash('Profile updated successfully!', 'success')
return redirect(url_for('profile.index'))

"""
import os
import sys
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

```

```

import argparse
import numpy as np

N_MFCC = 40

def extract_mfcc(path):
    import librosa
    y, sr = librosa.load(path, sr=22050, duration=5.0)
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=N_MFCC, n_fft=2048,
hop_length=512)
    return np.mean(mfcc.T, axis=0)[:N_MFCC]

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--data-dir', default='data/audio', help='Directory with real/ and
fake/ subdirs')
    parser.add_argument('--epochs', type=int, default=20)
    args = parser.parse_args()

    import tensorflow as tf
    from tensorflow.keras import layers, Model

    real_dir = os.path.join(args.data_dir, 'real')
    fake_dir = os.path.join(args.data_dir, 'fake')

    if not os.path.exists(real_dir) or not os.path.exists(fake_dir):
        print("Please create data directory structure:")
        print(" data/audio/real/ - authentic voice samples")
        print(" data/audio/fake/ - synthetic/deepfake voice samples")
        print("\nYou can use ASVspooof dataset or similar.")
        return

    X, y = [], []
    for label, d in [(0, real_dir), (1, fake_dir)]:
        for f in os.listdir(d):
            path = os.path.join(d, f)
            if not os.path.isfile(path):
                continue
            try:
                mfcc = extract_mfcc(path)
                if len(mfcc) == N_MFCC:

```

```

        X.append(mfcc)
        y.append(label)
    except Exception:
        pass

if len(X) < 20:
    print("Need at least 20 samples total. Add more audio files.")
    return

X = np.array(X).reshape(-1, N_MFCC, 1)
y = np.array(y)

idx = np.random.permutation(len(X))
split = int(0.8 * len(X))
X_train, X_val = X[idx[:split]], X[idx[split:]]
y_train, y_val = y[idx[:split]], y[idx[split:]]

inp = layers.Input(shape=(N_MFCC, 1))
x = layers.Conv1D(32, 3, activation='relu')(inp)
x = layers.MaxPooling1D(2)(x)
x = layers.Conv1D(64, 3, activation='relu')(x)
x = layers.GlobalAveragePooling1D()(x)
x = layers.Dense(32, activation='relu')(x)
x = layers.Dropout(0.3)(x)
x = layers.Dense(1, activation='sigmoid')(x)
model = Model(inputs=inp, outputs=x)
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

os.makedirs('models', exist_ok=True)
model.fit(X_train, y_train, validation_data=(X_val, y_val), epochs=args.epochs,
batch_size=16)
model.save_weights('models/audio_weights.keras')
print("Saved models/audio_weights.keras")
if __name__ == '__main__':
    main()
import os
import sys
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

import argparse
import numpy as np

```

```

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--data-dir', default='data/images', help='Directory with real/ and
fake/ subdirs')
    parser.add_argument('--epochs', type=int, default=10)
    parser.add_argument('--batch-size', type=int, default=16)
    parser.add_argument('--synthetic', action='store_true', help='Generate synthetic fake
images from real')
    args = parser.parse_args()

    import tensorflow as tf
    from tensorflow.keras.applications import EfficientNetB0
    from tensorflow.keras import layers, Model
    from tensorflow.keras.preprocessing.image import ImageDataGenerator

    IMG_SIZE = 224
    real_dir = os.path.join(args.data_dir, 'real')
    fake_dir = os.path.join(args.data_dir, 'fake')

    if args.synthetic and not os.path.exists(fake_dir):
        # Create synthetic fake images from real
        os.makedirs(real_dir, exist_ok=True)
        os.makedirs(fake_dir, exist_ok=True)
        print("Synthetic mode: place some real images in data/images/real/ then re-run.")
        print("Or download a dataset (e.g., FaceForensics++) and organize:")
        print(" data/images/real/ <- authentic images")
        print(" data/images/fake/ <- deepfake/manipulated images")
        return

    if not os.path.exists(real_dir) or not os.path.exists(fake_dir):
        print("Creating directories and generating synthetic data...")
        os.makedirs(real_dir, exist_ok=True)
        os.makedirs(fake_dir, exist_ok=True)
        # Download sample images
        try:
            from tensorflow.keras.utils import get_file

            url = "https://storage.googleapis.com/download.tensorflow.org/example_images/flower_photos.tgz"
            path = get_file(origin=url, extract=True)
            base = os.path.join(os.path.dirname(path), 'flower_photos')
            for cls in ['roses', 'daisy']:

```



```

src = os.path.join(base, cls)
if os.path.exists(src):
    for f in os.listdir(src)[:50]:
        src_f = os.path.join(src, f)
        if f.endswith('.jpg'):
            import shutil
            shutil.copy(src_f, os.path.join(real_dir, f))
            # Create fake: load, compress, save
            img = tf.keras.utils.load_img(src_f, target_size=(IMG_SIZE,
IMG_SIZE))

            arr = tf.keras.utils.img_to_array(img)
            fake_path = os.path.join(fake_dir, f)
            tf.keras.utils.save_img(fake_path, arr * 0.9 +
np.random.randn(*arr.shape) * 10)
            print("Generated synthetic dataset.")
except Exception as e:
    print("Could not auto-download. Please add images manually:")
    print(" data/images/real/ - authentic images")
    print(" data/images/fake/ - manipulated/deepfake images")
    print(str(e))
    return

datagen = ImageDataGenerator(
    rescale=1./255,
    shear_range=0.1,
    zoom_range=0.1,
    horizontal_flip=True,
    validation_split=0.2
)
train_gen = datagen.flow_from_directory(
    args.data_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=args.batch_size,
    class_mode='binary',
    subset='training'
)
val_gen = datagen.flow_from_directory(
    args.data_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=args.batch_size,
    class_mode='binary',
    subset='validation'
)

```

```

)

base = EfficientNetB0(weights='imagenet', include_top=False,
input_shape=(IMG_SIZE, IMG_SIZE, 3))
base.trainable = False
x = base.output
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(128, activation='relu')(x)
x = layers.Dropout(0.3)(x)
x = layers.Dense(1, activation='sigmoid')(x)
model = Model(inputs=base.input, outputs=x)
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

os.makedirs('models', exist_ok=True)
model.fit(train_gen, validation_data=val_gen, epochs=args.epochs)
model.save_weights('models/image_weights.keras')
print("Saved models/image_weights.keras")

if __name__ == '__main__':
    main()

```

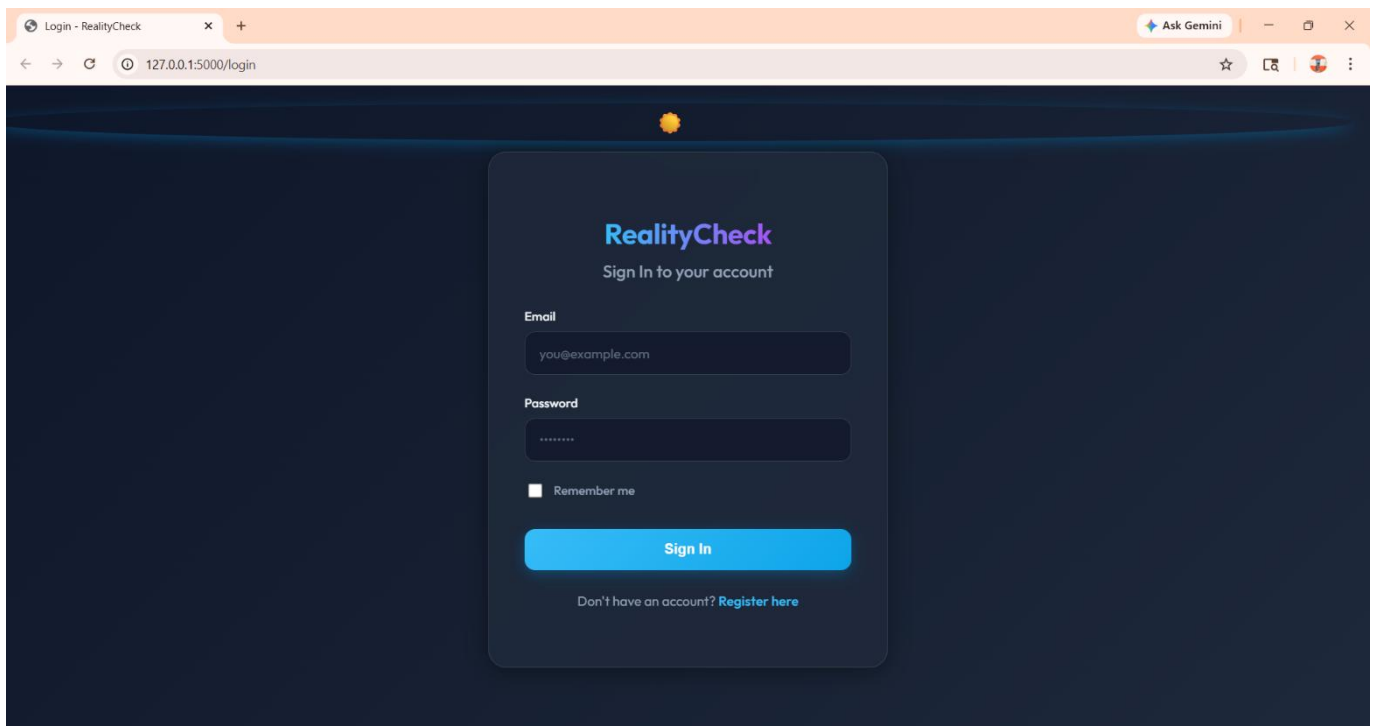
APPENDIX 2:

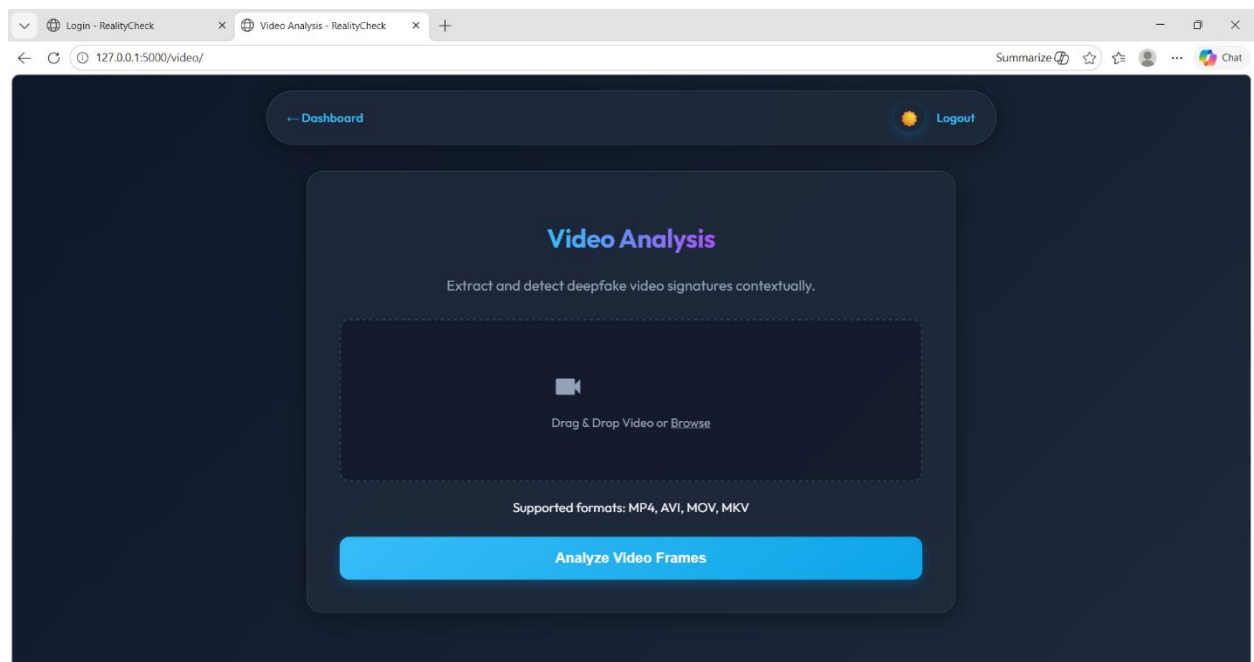
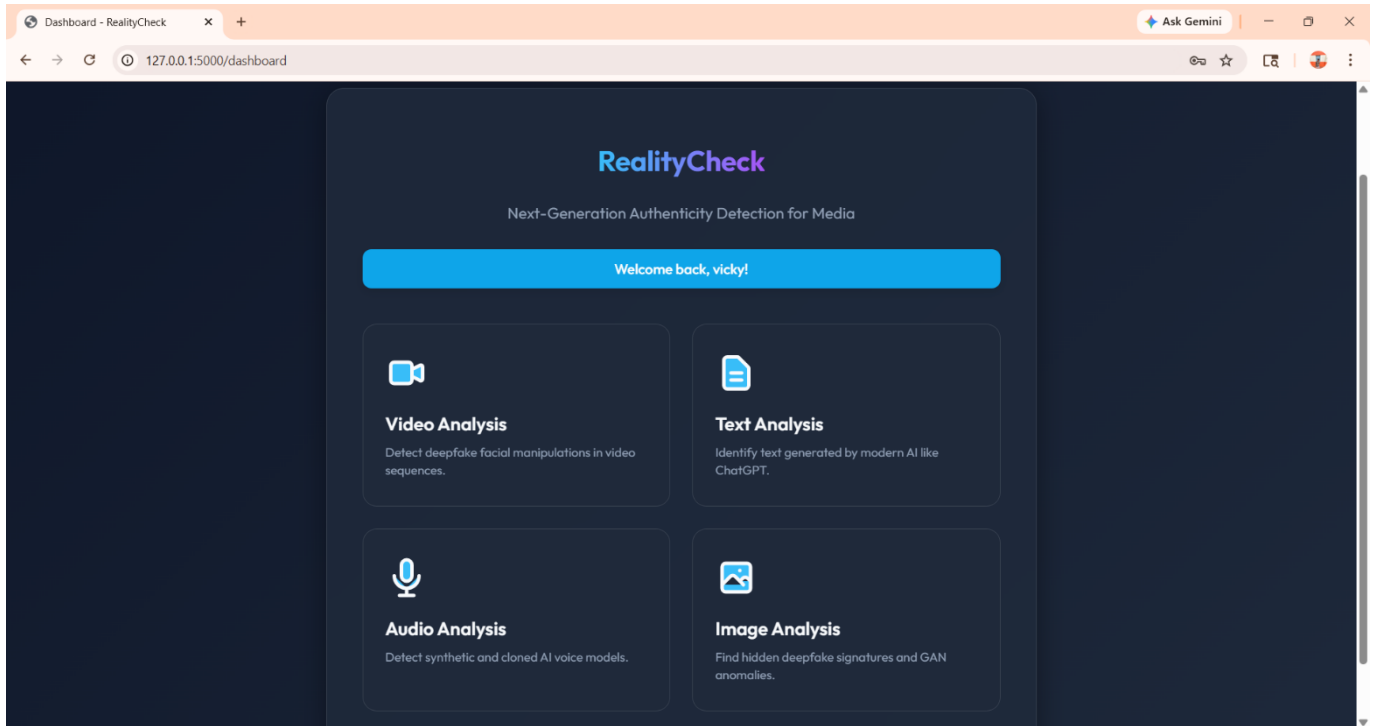
LIST OF PUBLICATION

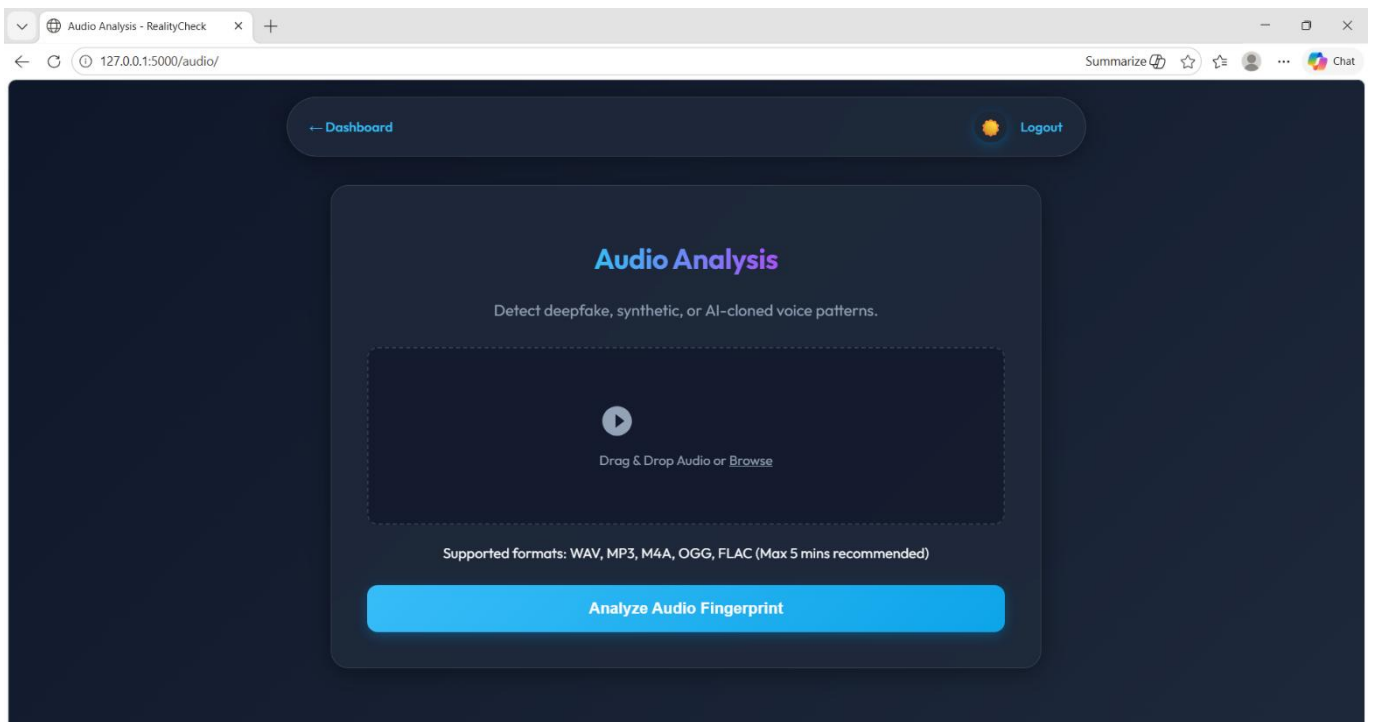
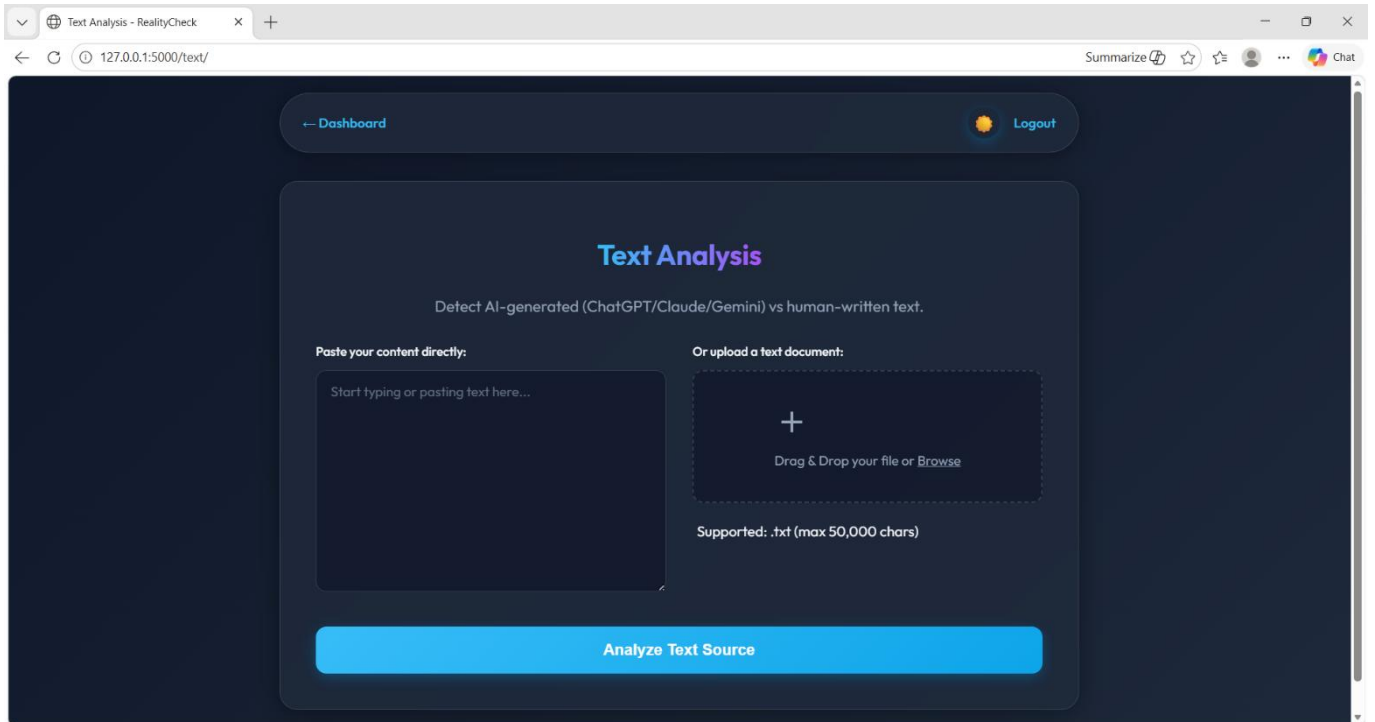
Karthikeyan S, Harshavarthan P, Saravanan V, Varshitha G, Varshika K, Vignesh T, Yuvaraj S, Manikkam V , “SMART TRAVEL COMPANION SYSTEM WITH BUDGET OPTIMIZATION USING RECOMMENDATIONS”, 7th International Conference on Artificial Intelligence and Smart Environments (ICAISE’25), held at Hammamet, Tunisia from November 6 to 8, 2025.

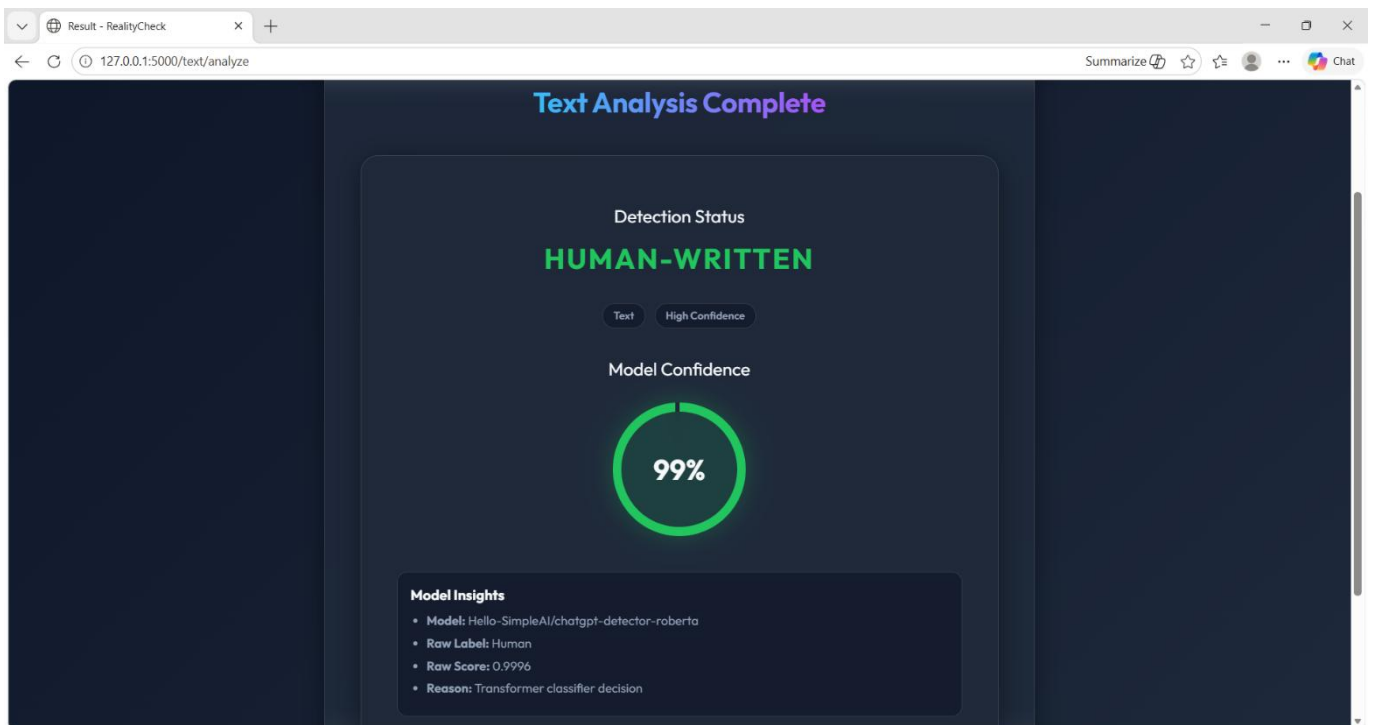
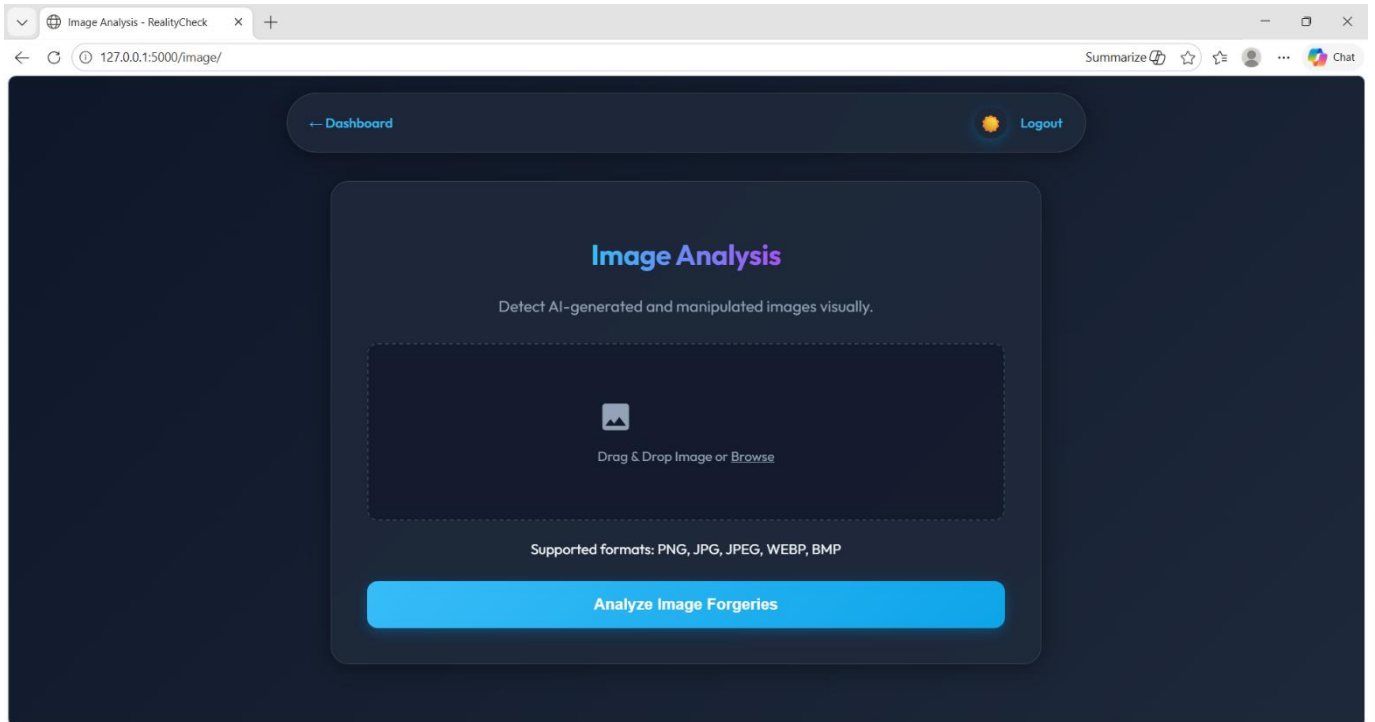
The image shows a Visual Studio Code editor window with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'REALITYCHECK_VIDEO' with various subfolders and files. The main editor window displays the 'app.py' file, which contains Python code for a Flask application. The code includes imports for Flask, extensions, routes, and models, followed by configuration settings and directory creation. The terminal window shows the command prompt 'PS D:\Realitycheck_Video>'.

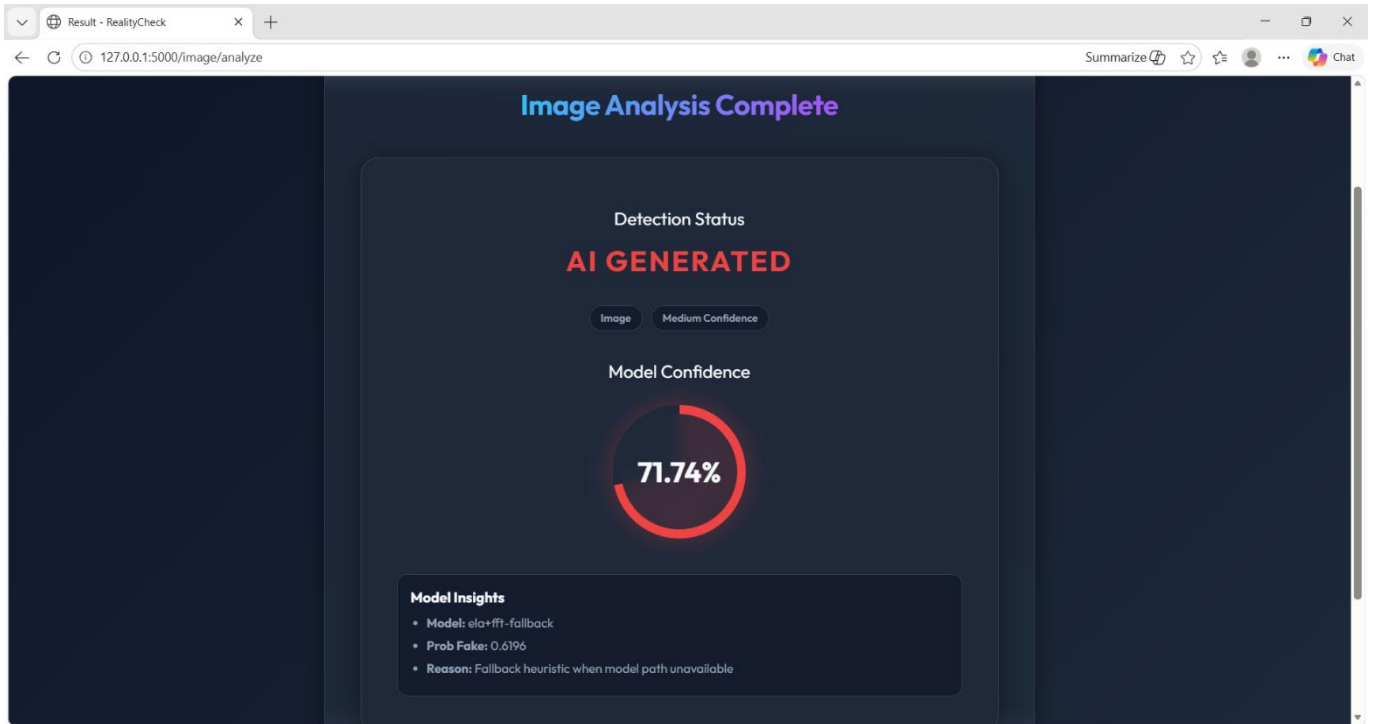
```
1 import os
2 from flask import Flask
3 from extensions import db, login_manager
4 from main_routes import main_blueprint
5 from auth_module.auth_routes import auth_blueprint
6 from auth_module.models import User
7 from video_module.video_routes import video_blueprint
8 from text_module.text_routes import text_blueprint
9 from audio_module.audio_routes import audio_blueprint
10 from image_module.image_routes import image_blueprint
11 from auth_module.profile_routes import profile_blueprint
12
13 app = Flask(__name__)
14 app.config['SECRET_KEY'] = os.environ.get('SECRET_KEY', 'dev-secret-key-change-in-production')
15 app.config['SQLALCHEMY_DATABASE_URI'] = os.environ.get('DATABASE_URL', 'sqlite:///realitycheck.db')
16 app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
17
18 app.config['UPLOAD_FOLDER'] = 'uploads/videos'
19 app.config['FRAME_FOLDER'] = 'uploads/frames'
20 app.config['AUDIO_FOLDER'] = 'uploads/audio'
21 app.config['IMAGE_FOLDER'] = 'uploads/images'
22 app.config['AVATAR_FOLDER'] = os.path.join('static', 'avatars')
23
24 os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)
25 os.makedirs(app.config['FRAME_FOLDER'], exist_ok=True)
26 os.makedirs(app.config.get('AUDIO_FOLDER', 'uploads/audio'), exist_ok=True)
27 os.makedirs(app.config.get('IMAGE_FOLDER', 'uploads/images'), exist_ok=True)
```

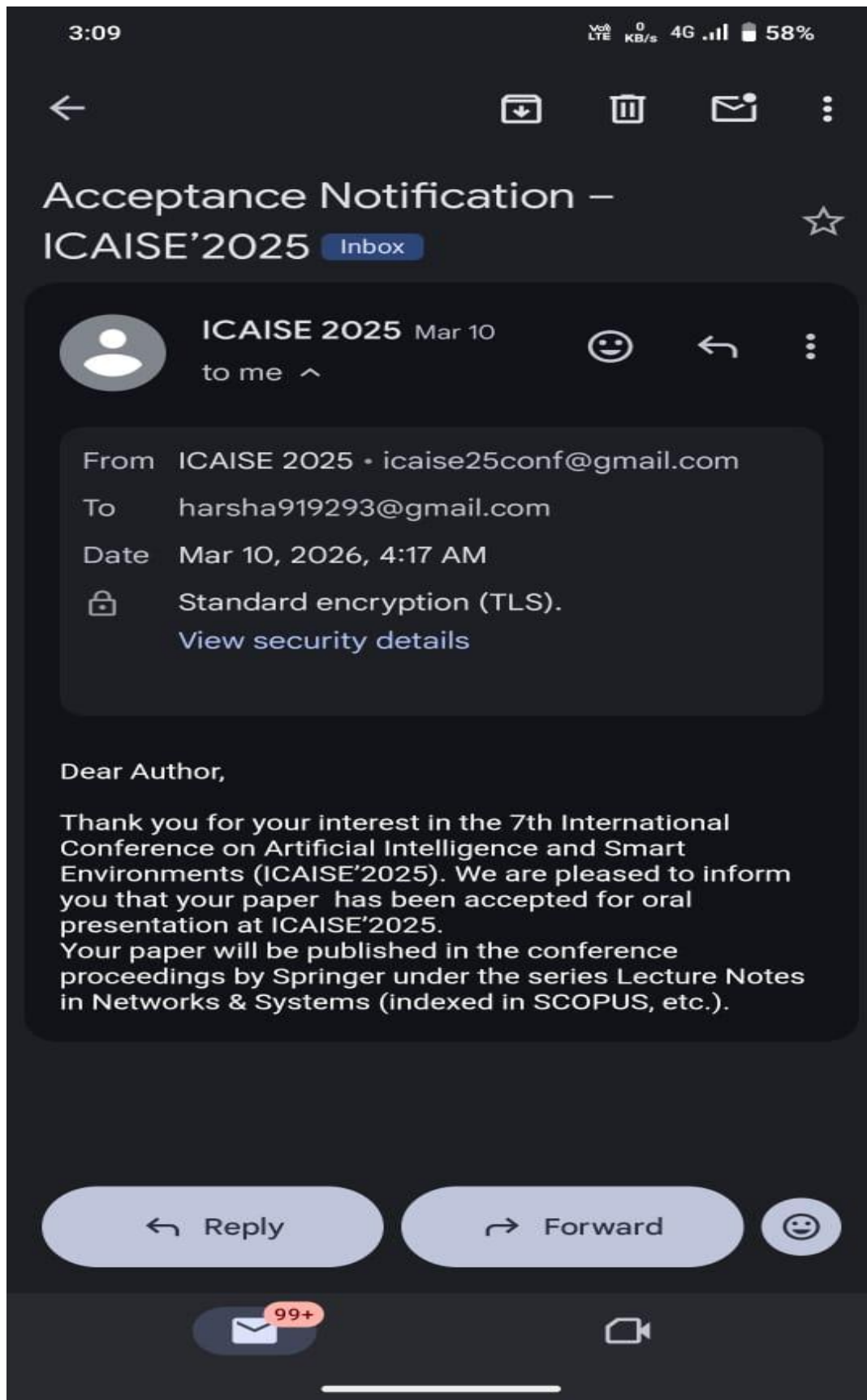














INTERNATIONAL CONFERENCE ON
ARTIFICIAL INTELLIGENCE & SMART ENVIRONMENTS

7th International Conference on
Artificial Intelligence and Smart Environments
November 06-08, 2025 Hammamet-Tunisia

Springer

Certificate OF PARTICIPATION

This certificate is proudly presented to

Manikkam V

UG Students, Department of Computer Science and Business Systems, Akshaya College of Engineering and Technology, Kinathukadavu, Coimbatore - 642 109, TamilNadu, India

for having attended the 7th International Conference on Artificial Intelligence and Smart Environments "ICAISE'2025", held from November 6 to 8, 2025, in Hammamet, Tunisia and presented his communication entitled:

" Smart Travel Companion System with Budget Op-timization Using Recommendations "

Co-authors: Harshavarthan P, Saravanan V, Varshitha G, Varshita K, Vignesh T, Karthikeyan S, Yuvaraj S
ID: 32

ICAISE'2025 General Chairs

Dr Ahmed JEMAL
ISET Sfax, Tunisia

Prof. Youssef FARHAOUI
FST Errachidia, Morocco



International Conference on Artificial Intelligence and Smart Environments
ICAISE'25
Hammamet, Tunisia
November 06-08
2025

